NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

ATTRIBUTE GRAMMARS

IN OCAMLFLAT/OFLAT

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
March, 2026

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ATTRIBUTE GRAMMARS

IN OCAMLFLAT/OFLAT

PEDRO EMANUEL INÁCIO BAILÃO

BSc in Computer Science and Engineering

Adviser: Artur Miguel Dias
Professor Auxiliar, DI-FCT, Universidade NOVA de Lisboa

Examination Committee

MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
March, 2026

Attribute Grammars
in OCamlFLAT/OFLAT

Copyright © Pedro Emanuel Inácio Bailão, NOVA School of Science and Technology, NOVA University Lisbon.
The NOVA School of Science and Technology and the NOVA University Lisbon have the right, perpetual and without geographical boundaries, to file and publish this dissertation through printed copies reproduced on paper or on digital form, or by any other means known or that may be invented, and to disseminate through scientific repositories and admit its copying and distribution for non-commercial, educational or research purposes, as long as credit is given to the author and editor.

This document was created with the (pdf/Xe/Lua)LaTeX processor and the NOVAthesis template (v7.1.29) [6].

"This is the way."

— Mandalorian

Abstract

Theoretical concepts in Formal Languages and Automata Theory (FLAT) are known for their formal, rigorous, and often complex nature, making them challenging to learn. To address this challenge, various tools have been developed over the years, allowing users to engage with these concepts through practical exercises and examples.

At FCT-UNL, two pedagogical tools were started to aid in this learning process: the OCamlFLAT library, which provides logical support for FLAT concepts, and the OFLAT graphical application, a web-based tool offering interactive visualizations of these ideas.

The first goal of this dissertation is to extend the OCamlFLAT library by incorporating support for attribute grammars. The second goal is to enhance the OFLAT graphical application by adding features that allow for the visualization and interaction with these grammar concepts.

A key focus of this work is the pedagogical enhancement of the developed software. This dissertation explains some theoretical concepts of Theory of computation, emphasizing the topic of attribute grammars, discusses some of the best existing software applications for Theory of computation concepts, describes the current state of OCamlFLAT/OFLAT, presents the main tools that are going to be used in the project and finishes with a work plan.

Keywords: Formal Languages, Automata Theory, OCaml, OFLAT, OCamlFLAT, Functional Programming, Formal Grammars

Resumo

Os conceitos teóricos em Teoria de Linguagens Formais e Autómatos (FLAT) são conhecidos pela sua natureza formal, rigorosa e muitas vezes complexa, o que os torna desafiantes para aprender. Para mitigar essa dificuldade, várias ferramentas foram desenvolvidas ao longo dos anos, permitindo que os utilizadores interajam com esses conceitos através de exercícios e exemplos práticos.

Na FCT-UNL, foram criadas duas ferramentas pedagógicas com este propósito: a biblioteca OCamlFLAT, que fornece suporte lógico para os conceitos de FLAT, e a aplicação gráfica OFLAT, uma ferramenta web que oferece visualizações interativas desses conceitos.

O primeiro objetivo desta dissertação é estender a biblioteca OCamlFLAT, incorporando suporte para gramáticas com atributos. O segundo objetivo é melhorar a aplicação gráfica OFLAT, adicionando funcionalidades que permitam a visualização e interatividade com estes conceitos de gramáticas.

O foco principal deste trabalho é a melhoria pedagógica do software desenvolvido.

Esta dissertação explica alguns conceitos teóricos da Teoria da Computação, com ênfase no tema das gramáticas com atributos, discute algumas das melhores aplicações de software existentes para conceitos da Teoria da Computação, descreve o estado atual do OCamlFLAT/OFLAT, apresenta as principais ferramentas que serão utilizadas no projeto e finaliza com um plano de trabalho.

Palavras-chave: Teoria de Linguagens Formais, Autómatos, OCaml, OFLAT, OCamlFLAT, Programação Funcional, Gramáticas Formais

Contents

List of Figures **xiii**

1 Introduction **1**
1.1 Context 1
1.2 Objectives 1
1.3 Document Structure 1

2 Theory of Computation **3**
2.1 Chomsky Hierarchy 3
2.1.1 Type 0: Unrestricted Grammars 3
2.1.2 Type 1: Monotonic Grammars 4
2.1.3 Type 2: Context-Free Grammars 5
2.1.4 Type 3: Regular Grammars 5
2.2 Attribute Grammars 6
2.2.1 Synthesized Attributes 6
2.2.2 Inherited Attributes 7
2.2.3 Conditions 8
2.2.4 Circularity 10
2.3 Automata 12
2.3.1 Types of Automata 12

3 Grammar Tools **15**
3.1 JFLAP 15
3.2 CGM 16
3.3 ANTLR 16
3.4 YACC 17

4 OCamlFLAT/OFLAT **19**
4.1 OCamlFLAT 19
4.2 OFLAT Overview 20

4.3 OFLAT User Interface 20

5 Technology **23**
5.1 Programming with OCaml 23
5.1.1 Declarative Programming 24
5.2 OCaml/JavaScript Interoperability 24
5.2.1 JavaScript Types 24
5.2.2 Bindings 25
5.2.3 `js_of_ocaml` Modules 25
5.3 Cytoscape.js 25

6 OCAML-FLAT - Implementation **27**
6.1 Data types created for attribute grammars in Ocaml 29
6.2 The `calcAttributes` fuction 30
6.2.1 Inputs and Output 30
6.2.2 Evaluation Procedure at an Internal Node 31
6.2.3 Deterministic Order and Rationale 31
6.2.4 Error Handling 32
6.3 Auxiliary Methods Used by `calcAttributes` 32
6.3.1 `eval_children_left_to_right` 32
6.3.2 Role of the Auxiliary `loop` Function 34
6.3.3 Step-by-Step Operation of `loop` 34
6.3.4 The Loop Necessity 35
6.3.5 Relation Back to `calcAttributes` 36
6.3.6 `getRootRule` 36
6.3.7 `apply_equations_at_head` 37
6.3.8 `check_conditions_at_node` 38
6.4 The `accept` fuction 38
6.5 The `validate` function 38
6.5.1 Auxiliary Methods Used by `validate` 40
6.6 Cycle Detection 40
6.6.1 `build_dep_graph` 41
6.6.2 `has_cycle_graph` 42
6.7 Unit Tests: Trees and Attribute Annotations 42
6.7.1 `test_ag1_simple_synthesized` (`ag1`, tree `pt1`) 43
6.7.2 `test_parseTree` (Grammar `ag2`, word `3+2+9~`) 43
6.7.3 `test_ag2_simple_synthesized_and_inherited` (`ag2`, tree `pt2`) . 44
6.7.4 `test_ag3_simpler_synthesized_and_inherited` (`ag3`, tree `pt3`) 45
6.7.5 `test_ag4_type_mismatch` (`ag4`, tree `pt4`) 46
6.7.6 `test_ag5_div_zero` (`ag5`, tree `pt5`) 46
6.7.7 `test_ag6_lattr_violation` (`ag6`, tree `pt6`) 47

6.7.8 test_ag7_default_index (ag7, tree pt7) 47
6.7.9 test_ag8_boolean_flags (ag8, tree pt8) 48
6.7.10 test_ag9_list_length (ag9, tree pt9=make_list 30) 48
6.7.11 test_ag10_inherited_default (ag10, tree pt10) 49
6.7.12 test_ag11_out_of_range_ref (ag11, tree pt11) 50
6.7.13 test_has_cycles_ok (ok_ag) 50
6.7.14 test_has_cycles_detected (cyc_ag) 50
6.7.15 test_accept_with_tree_ok 51
6.7.16 test_accept_words 51
6.7.17 test_validate 52
6.7.18 test_ag_expr_ext (ag_expr_ext, tree pt_expr_ext) 52

7 Evaluation and future work **55**
7.1 Evaluation 55
7.2 Conclusion 55

Bibliography **57**

List of Figures

2.1 Chomsky hierarchy 4
2.2 Attribute grammar for signed binary numbers [4] 10
2.3 Attribute dependency graph for the grammar in figure 2 [4] 10
2.4 Modified attribute dependency graph with circularity 11

4.1 Current Version OFLAT User Interface 20
4.2 OFLAT User Interface Division. Red - Sidebar, Blue - Workspace 21
4.3 Workspace Closeup 22
4.4 Workspace Window Split 22

1

Introduction

1.1 Context

The study of Formal Languages and Automata Theory (FLAT) is integral to Computer Engineering and Computer Science programs. However, due to its abstract and mathematical nature, students often struggle to fully grasp the concepts and models involved. Therefore, a tool that complements the study of FLAT theory by being practical, user-friendly, and visually oriented is highly beneficial.

In recent years, the OCamlFLAT/OFLAT tool has been developed at FCT-UNL, offering extensive support in this field. Despite its advanced state, with support for several key models, there are still many features that could enhance its functionality.

This dissertation aims to introduce new capabilities to OCamlFLAT/OFLAT, enhancing both the OCamlFLAT library and the OFLAT graphical interface.

1.2 Objectives

The primary objective of this dissertation is to incorporate support for attribute grammars. Attribute grammars extend context-free grammars by allowing additional information (attributes) to be associated with production rules. These attributes are crucial for describing the semantic properties of the languages generated by the grammars.

To achieve this, the OCamlFLAT library needs to be extended to support attribute grammars. Additionally, graphical enhancements are required to enable the creation and editing of these grammars within the OFLAT platform.

1.3 Document Structure

This thesis is structured into six chapters:

- **Introduction**: Provides an overview of the dissertation's context, objectives, and structure.

- **Theory of Computation**: Lays the theoretical foundation for the study of FLAT models. It begins with an exploration of the Chomsky hierarchy and progresses into discussions of the four language types (Type 0, 1, 2, and 4).
- **Grammar Tools**: Examines existing pedagogical tools that support the teaching and learning of FLAT theory, with a focus on three prominent systems: JFLAT, ANTLR, and CGM.
- **OCamlFLAT and OFLAT Systems**: Introduces the OCamlFLAT library and the OFLAT platform.
- **Technology**: Covers the technical aspects of developing the OCamlFLAT/OFLAT systems.
- **Planning**: In this chapter we define and scheduled the tasks to be accomplished in this project.

2

Theory of Computation

In Theory of Computation, various models of computation with different levels of expressive power are studied. To discuss and compare different levels of expressive power, it is inevitable to consider algorithms in the context of some application domain. The typically chosen application domain is the syntax of formal languages. There are two techniques for defining languages that are typically used. The first technique is based on the concept of generation and uses the formalism of grammars. Another technique is based on the mechanism of recognition and uses the formalism of automata.

2.1 Chomsky Hierarchy

Noam Chomsky (with the help of Marcel-Paul Schützenberger) identified several classes of formal languages, with different levels of specification complexity, requiring computational models with different levels of expressive power to solve the problem of recognizing these languages [2].

The Chomsky hierarchy involves four levels of language types. Each type of language is associated with a type of generator(grammar) and a type of recognizer(automata).

- Type 0 — Unrestricted Grammars
- Type 1 — Context-Sensitive Grammars
- Type 2 — Context-Free Grammars
- Type 3 — Regular Grammars

2.1.1 Type 0: Unrestricted Grammars

Type 0 grammars, also known as unrestricted grammars, are the most general class in the Chomsky hierarchy. They have almost no restrictions on their production rules, meaning that any string of symbols can be replaced by any other string of symbols. This flexibility allows Type 0 grammars to generate any language that is recognizable by a Turing machine,

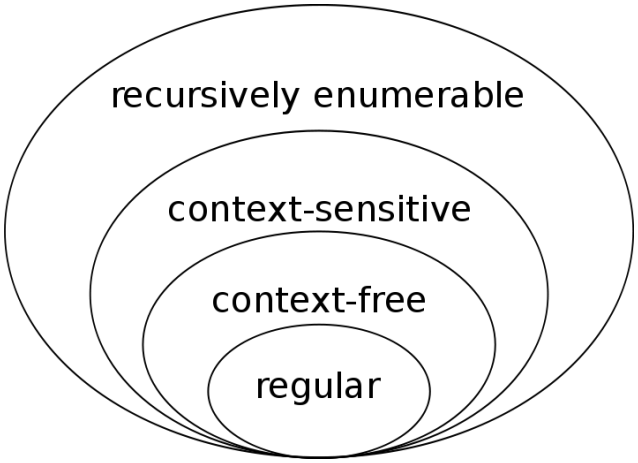


Figure 2.1: Chomsky hierarchy

making them equivalent in power to Turing machines. In other words, they can describe all recursively enumerable languages [2].

An example of an unrestricted grammar is:

$$G = (\{S, A, B\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow aAB, AB \rightarrow BA, A \rightarrow a, B \rightarrow b, B \rightarrow \lambda\}$$

Derivation of the word "aba":

$$\begin{aligned} S &\rightarrow aAB \\ aAB &\rightarrow aBA \\ aBA &\rightarrow abA \\ abA &\rightarrow aba \end{aligned}$$

2.1.2 Type 1: Monotonic Grammars

On type 1 grammars, or monotonic grammars, the length of the string on the left-hand side of a production rule is less than or equal to the length of the string on the right-hand side. This property ensures that the derivation process does not shorten the string. Type 1 grammars can generate languages that are recognized by a linear bounded automaton, which is a Turing machine with a tape of limited length proportional to the input size [2].

An example of a monotonic grammar is:

$$G = (\{S, B\}, \{a, b, c\}, P, S)$$

$$P = \{S \rightarrow aBSc, S \rightarrow abc, Ba \rightarrow aB, Bb \rightarrow bb\}$$

Derivation of the word "aaabbbccc":

$$
\begin{aligned}
S &\longrightarrow aBSc \\
aBSc &\longrightarrow aBaBScc \\
aBaBScc &\longrightarrow aBaBabccc \\
aBaBabccc &\longrightarrow aBaaBbccc \\
aBaaBbccc &\longrightarrow aaBaBbccc \\
aaBaBbccc &\longrightarrow aaaBBbccc \\
aaaBBbccc &\longrightarrow aaabbbccc
\end{aligned}
$$

2.1.3 Type 2: Context-Free Grammars

On type 2 grammars, or context-free grammars, the left-hand side consists of a single non-terminal symbol, and the right-hand side can be any string of terminal and/or non-terminal symbols. These grammars allow context-free grammars to generate languages that can be recognized by a pushdown automaton [2].

An example of a context-free grammar is:

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \longrightarrow aSb, S \longrightarrow ab\}$$

Derivation of the word "aabb":

$$
\begin{aligned}
S &\longrightarrow aSb \\
aSb &\longrightarrow aaSbb \\
aaSbb &\longrightarrow aabb
\end{aligned}
$$

2.1.4 Type 3: Regular Grammars

On type 3 grammars, also known as regular grammars, represent the most constrained category within the Chomsky hierarchy. The production rules for these grammars are quite specific, each rule's left-hand side is a single non-terminal symbol, while the right-hand side is either a single terminal symbol or a terminal symbol followed by a single non-terminal symbol. This strict format allows regular grammars to generate languages that can be recognized by finite automata. [2].

An example of a regular grammar is:

$$G = (\{S\}, \{a, b\}, P, S)$$

$$P = \{S \longrightarrow aS, S \longrightarrow b\}$$

Derivation of the word "aaab":

$$S \rightarrow aS$$
$$aS \rightarrow aaS$$
$$aaS \rightarrow aaaS$$
$$aaaS \rightarrow aaab$$

This dissertation will focus on an extension of type 2 grammars: Attribute Grammars.

2.2 Attribute Grammars

Attribute grammars are an extension of context-free grammars that associate attributes with the grammar symbols and define rules for computing these attributes, in addition, add conditions that are boolean expressions using the attributes and are used to restrict the generated language. These features provide a way to carry semantic information through the parse tree, enabling context-sensitive analysis [5].

There are two types of attributes:

- **Synthesized Attributes**: These are computed from the attributes of the children nodes in the parse tree. They propagate information up the tree.
- **Inherited Attributes**: These are computed from the attributes of the parent and/or sibling nodes. They propagate information down and across the tree.

By extending context-free grammars with attributes and conditions, attribute grammars enable more powerful and flexible language descriptions. In fact, it is easy to integrate contextual aspects into a grammar, often more intuitively than through a monotonic grammar. For example, for the language of sequences of 'a's with a length less than or equal to ten, instead of needing 10 rules, we can use an attribute and some conditions to reach the same result.

2.2.1 Synthesized Attributes

Synthesized attributes are computed from the attributes of the children of a node in the derivation tree. These attributes propagate information up the tree, allowing for the evaluation of properties that depend on the structure of the subtree rooted at a particular node [5].

For example, consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$
$$P = \{S \rightarrow A, A \rightarrow aA|b\}$$

We can define a synthesized attribute length that calculates the length of the generated string. The rules for computing this attribute are as follows:

$$\text{length}(S) = \text{length}(A)$$

$$\text{length}(A \rightarrow aA) = 1 + \text{length}(A)$$

$$\text{length}(A \rightarrow b) = 1$$

Derivation of the word "aaab" along with the simultaneous computation of the attribute:

$$S \Rightarrow A, \quad \text{length}(S) = \text{length}(A)$$

$$A \Rightarrow aA, \quad \text{length}(A) = 1 + \text{length}(A)$$

$$aA \Rightarrow aaA, \quad \text{length}(A) = 1 + (1 + \text{length}(A))$$

$$aaA \Rightarrow aaaA, \quad \text{length}(A) = 1 + (1 + (1 + \text{length}(A)))$$

$$aaaA \Rightarrow aaab, \quad \text{length}(A) = 1$$

Back-substituting:

$$\text{length}(A) = 1$$
$$\text{length}(A) = 1 + 1 = 2$$
$$\text{length}(A) = 1 + 2 = 3$$
$$\text{length}(A) = 1 + 3 = 4$$

Thus, the length of the generated string **aaab** is:

$$\text{length}(S) = 4$$

2.2.2 Inherited Attributes

Inherited attributes are passed from parent nodes to child nodes in the derivation tree. These attributes propagate information down and across the parse tree [5].

For example, consider the following grammar:

$$G = (\{S, A\}, \{a, b\}, P, S)$$

$$P = \{S \rightarrow A, A \rightarrow aA|b\}$$

We can define an inherited attribute word that accumulates the word that is being generated. of the generated string. The rules for computing this attribute are as follows:

$$\text{word}(S) = ""$$

$$\text{word}(A \rightarrow aA) = \text{word}(A) + "a"$$

$$\text{word}(A \rightarrow b) = \text{word}(A) + "b"$$

Derivation of the word "aaab" along with the simultaneous computation of the attribute:

$$S \Rightarrow A, \quad \text{word}(S) = ""$$

$$A \Rightarrow aA, \quad \text{word}(A) = \text{word}(A) + "a"$$

$$aA \Rightarrow aaA, \quad \text{word}(A) = \text{word}(A) + "a"$$

$$aaA \Rightarrow aaaA, \quad \text{wordx}(A) = \text{word}(A) + "a"$$

$$aaaA \Rightarrow aaab, \quad \text{word}(A) = \text{word}(A) + "b"$$

Back-substituting:

$$\begin{aligned}
&\text{word}(S) = ""\\
&\text{word}(A) = \text{word}(S) + "a" = "a"\\
&\text{word}(A) = "a" + "a" = "aa"\\
&\text{word}(A) = "aa" + "a" = "aaa"\\
&\text{word}(A) = "aaa" + "b" = "aaab"
\end{aligned}$$

Thus, the accumulated word of the generated string is:

$$\text{word}(S) = "aaab"$$

2.2.3 Conditions

This section was inspired by [1]. Consider the language of strings of the form $a^n b^n c^n$, where the number of 'a's, 'b's, and 'c's must be equal. A context-free grammar can generate strings with the correct structure but cannot enforce the equal lengths of the sequences. But as we saw earlier, we can add conditions to an attribute grammar, ensuring this requirement is met, as we can see in the example below.

<letter_sequence> ::= <asequence> <bsequence> <csequence>

condition: Size(<asequence>) = Size(<bsequence>) = Size(<csequence>)

<asequence> ::= a

Size(<asequence>) ← 1

| <asequence> a

Size(<asequence>) ← Size(<asequence>) + 1

<bsequence> ::= b

Size(<bsequence>) ← 1

| <bsequence> b

Size(<bsequence>) ← Size(<bsequence>) + 1

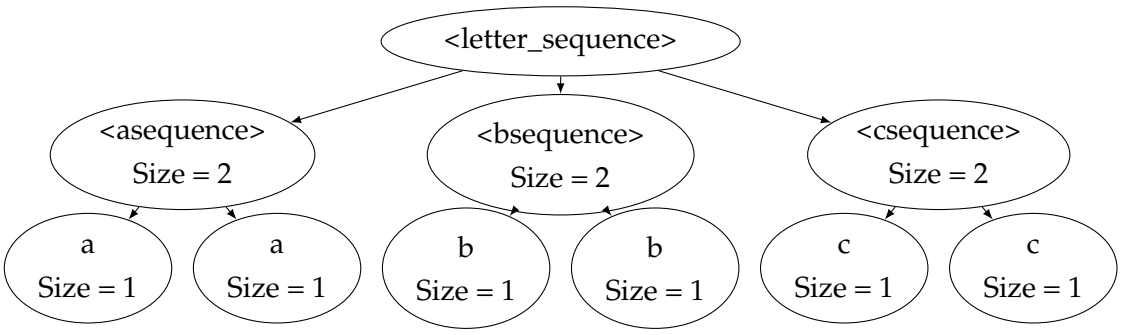
<csequence> ::= c

Size(<csequence>) ← 1

| <csequence> c

Size(<csequence>) ← Size(<csequence>) + 1

If we now parse the string "aabbcc", we get the following parse tree:



As we can see this tree shows the Size attribute is synthesized up the tree, ensuring that the length of the 'a's, 'b's, and 'c's sequences are equal. And then on the root we check the following condition:

Size(<asequence>) = Size(<bsequence>) = Size(<csequence>)

3 = 3 = 3

2.2.4 Circularity

For an attribute grammar to be well-formed, the attributes associated with nonterminals at any node in the parse tree must be evaluable based on the grammar's semantic rules [4]. Since non-trivial context-free grammars can generate an infinite number of parse trees, determining well-formedness is a complex task [5]. The primary challenge lies in circular dependencies within the attribute dependency graph. For instance, if attribute a1 depends on a2, which in turn depends on a1, neither attribute can be evaluated, leading to an evaluation deadlock.

Consider the following grammar in figure 2.2

0: Number → Sign List	(i) List↓Scale := 0
	(ii) Number↑Value := IF Sign↑Neg THEN - List↑Value ELSE List↑Value
1: Sign → +	(i) Sign↑Neg := False
2: Sign → -	(i) Sign↑Neg := True
3: List → BinaryDigit	(i) BinaryDigit↓Scale := List↓Scale
	(ii) List↑Value := BinaryDigit↑Value
4: $List_0$ → $List_1$ BinaryDigit	(i) $List_1$↓Scale := $List_0$↓Scale + 1
	(ii) BinaryDigit↓Scale := $List_0$↓Scale
	(iii) $List_0$↑Value := $List_1$↑Value + BinaryDigit↑Value
5: BinaryDigit → 0	(i) BinaryDigit↑Value := 0
6: BinaryDigit → 1	(i) BinaryDigit↑Value := $2^{\text{BinaryDigit↓Scale}}$

Figure 2.2: Attribute grammar for signed binary numbers [4]

The best way to visualize the circularity is to actually draw the dependency graph for the grammar. Figure 3 shows such a graph for the attribute grammar in figure 2.2.

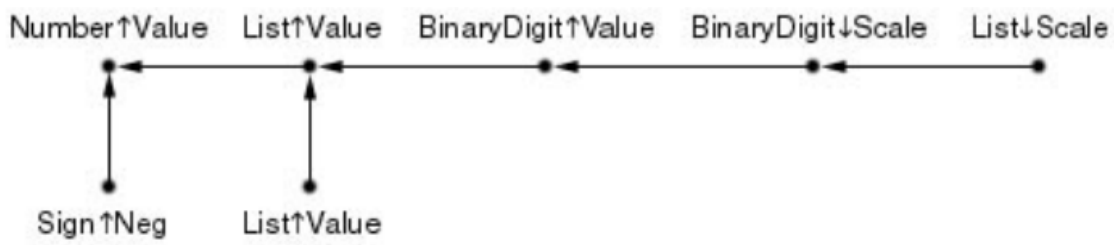


Figure 2.3: Attribute dependency graph for the grammar in figure 2 [4]

The dependency graph is constructed by combining the individual dependencies from each rule in the grammar, which explains the presence of multiple List↑Value attributes. In this case, the graph contains no circular dependencies, meaning it is well-formed—i.e., all possible parse trees generated by the grammar can be evaluated using the semantic rules. A circular dependency could be introduced by reversing the arrow from Sign↑Neg to Number↑Value and adding an arrow from Sign↑Neg to the lower List↑Value as we can see in the figure below. In such a scenario, none of the four attributes forming the cycle

could be evaluated. The challenge in attribute grammars is detecting these cycles within the attribute dependency graph[4].

Knuth characterizes noncircular attribute grammars by lifting the local dependencies stated in each production's semantic rules to a global dependency relation over attribute occurrences. For a production $A \rightarrow \alpha$, one records which attributes on the right-hand side flow into which attributes on the left (and vice versa for inherited attributes). Aggregating these per-production relations and taking their transitive closure yields a summarized graph over pairs (N, attr). The test is simple: if the closure contains a cycle that returns to the same (N, attr), the specification is circular; if no such cycle exists, the grammar is noncircular and every parse tree admits an evaluation order consistent with the dependencies [4, 5].

Several practical optimizations make cycle testing and evaluation efficient. The dependency graph can be decomposed into connected components and analyzed component wise, with early termination as soon as a back-edge closes a cycle. Transitive closure can be implemented with bit-set dataflow or fixed-point iteration, exploiting sparsity to reduce cost. Partitioning attributes by kind (synthesized versus inherited) often shrinks the graph without changing outcomes, and restricting specifications to S-attributed, L-attributed, or ordered attribute grammars yields subclasses with trivial or linear evaluation schedules. In evaluators and compiler generators, visit schedules and demand-driven or incremental evaluation avoid unnecessary computations while respecting the same dependency relation [4, 5].

When a cycle is reported, common remedies are to break mutual dependencies by rephrasing rules (e.g., moving information from inherited to synthesized or introducing a carrier attribute that defers computation), to refactor productions that mix context propagation with value computation, or to precompute context outside the cycle. Tools typically print a minimal cycle as a sequence of (N, attr) links that helps localize the cause; keeping a diagram of the dependency graph alongside the grammar makes validation and revision straightforward. Once noncircularity is established, attributes can be scheduled in a topological order that aligns with parser actions, ensuring evaluation terminates without deadlocks [4, 5].

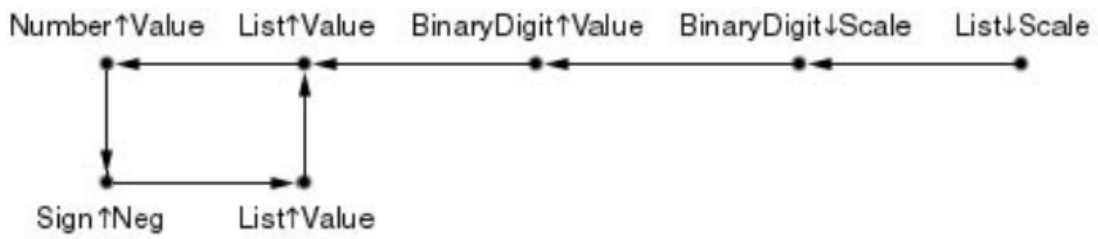


Figure 2.4: Modified attribute dependency graph with circularity

2.3 Automata

Automata theory investigates abstract models of computation and the formal languages they characterize [9]. In the Chomsky hierarchy, language classes align with canonical machine models.

2.3.1 Types of Automata

2.3.1.1 Finite Automata

A (deterministic) finite automaton (FA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of states, Σ is the input alphabet, $\delta : Q \times \Sigma \to Q$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of accepting states. FAs recognize exactly the regular languages. On input $w \in \Sigma^*$, the automaton consumes symbols left to right; if the final state lies in F, w is accepted.

2.3.1.2 Turing Machines

A (single-tape) Turing machine (TM) has a finite control, a bi-infinite tape over a work alphabet $\Gamma \supseteq \Sigma$, a head that reads/writes/moves, and a transition function δ. TMs recognize the recursively enumerable languages and decide the recursive languages, providing a formal basis for computability theory [5].

2.3.1.3 Linear Bounded Automata

A linear bounded automaton (LBA) is a non-deterministic TM whose head never moves outside the tape segment initially occupied by the input; equivalently, its space use is $O(n)$ for input length n. LBAs recognize exactly the context-sensitive languages and are strictly more powerful than PDAs but weaker than unrestricted TMs [5].

2.3.1.4 Pushdown Automata

A pushdown automaton (PDA) augments an FA with a stack. Formally, a PDA can be given as a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where Γ is the stack alphabet, $Z_0 \in \Gamma$ is the initial stack symbol, and δ can read an input symbol (or ϵ) and the stack top, possibly push/pop, and move to a new state. PDAs recognize exactly the context-free languages, enabling nested structures such as balanced parentheses [5].

Attribute grammars are implemented on top of the parser's pushdown stack. Each stack entry carries the grammar symbol (and, in LR parsers, the state) together with an attribute record. A *shift* pushes the next token along with its lexical attributes; a *reduce* by $A \to X_1 \cdots X_k$ pops the X_i entries, applies the production's semantic rule to compute A's attributes, and pushes A back with those values. LL parsers realize the same idea via the call stack: inherited attributes are passed as parameters to recursive procedures,

and synthesized attributes are returned as results. Single pass evaluation is ensured for S attributed (only synthesized) and L-attributed (controlled inherited flow) specifications.

For arithmetic expressions, let the token *num* carry a lexical value. Reductions compute a synthesized attribute `.val`: $F \rightarrow num$ yields $F.val := num.lexval$; $T \rightarrow T * F$ computes $T.val := T_1.val \times F.val$; $E \rightarrow E + T$ gives $E.val := E_1.val + T.val$; and $E \rightarrow T$ sets $E.val := T.val$. In declarations, inherited attributes convey the current type from a specifier to each identifier in a list, while synthesized attributes propagate information upward. The PDA continues to recognize the context-free structure, and the attribute rules fire at well defined parser actions in lockstep with the parse.

3

Grammar Tools

The tools for FLAT can be classified into two major categories: text-based automaton simulators and graphical interface-based automaton simulators. The first involves specifying a model through structured text, which must be processed using a parser, the second uses a graphical interface that allows creating, editing, and using an automaton to recognize words

The different existing tools can also support only one type of automaton or multiple types of automata, such as finite automata, pushdown automata, and Turing machines. There are also tools that deal with regular expressions and grammars, and in such cases, it is common for conversion operations to be available between automata and vice versa.

While *attribute grammars* extend CFGs with synthesized/inherited attributes to capture context-sensitive semantics (e.g., type information, name binding), the ecosystem lacks a JFLAP/CGM-style, classroom-ready GUI for experimenting with them. Available AG toolchains such as *ANTLR* and *YACC* are powerful and used in compiler research, but they expect research to write attribute equations/specifications rather than manipulate structures visually; thus they are not aimed at introductory FLAT workflows.

Next, some tools are presented and classified according to the indicators described above, i.e., whether they are text-based tools or graphical interface-based tools; whether they implement only one type of automaton or multiple types, and whether they support regular expressions and grammars. We will discuss three such tools: **JFLAP**, **CGM**, **ANTLR**, and **YACC**.

3.1 JFLAP

JFLAP (Java Formal Languages and Automata Package) is a long-running teaching tool developed at Duke University under Susan H. Rodger that lets students *build*, *visualize*, and *experiment* with core models from formal languages and automata theory. In practice, it covers nondeterministic finite automata (NFA/DFA), nondeterministic pushdown automata (PDA), multi-tape Turing machines and several classes of grammars.

JFLAP is useful when we move beyond drawing states and arrows and use the built-in

construction and *proof* helpers. For regular languages, you can take a regular expression, generate an ε-NFA, convert to a DFA, minimize it, and then push the result back to an equivalent regular expression or regular grammar; the intermediate steps are visible and repeatable. For context-free languages, you can work with CFGs and PDAs and run the standard transformations between them; these were emphasized in the Duke materials that originally motivated JFLAP's design. On the Turing side, the tool supports single-tape and multi-tape machines (2 to 5 tapes) and provides a clear step through view of each head, making classic exercises (e.g., $L = a^n b^n c^n$ with three tapes) far less opaque to learners.

Given the expression $(ab \mid ba)^*$, users can: (i) enter the RE in JFLAP's RE editor; (ii) auto-generate an ε-NFA; (iii) convert to a DFA; (iv) minimize; and (v) optionally convert the minimized DFA back to an RE for a check of equivalence.

3.2 CGM

CGM (Context-Free Grammar Manipulator) is a lightweight, GUI-based environment from the University of Porto for experimenting with *context-free grammars* (CFGs). It targets classroom use: users can (i) edit a CFG and compile it; (ii) transform it to CNF with the usual clean-ups (remove ϵ-rules, unit rules, and useless symbols, then binarize productions); (iii) parse input strings; and (iv) construct and inspect parse trees.

This tool implements a bottom-up *CYK* parser: it first converts the user grammar to CNF, then fills a dynamic-programming table $N[i, j]$ with the nonterminals that derive each substring, and finally reconstructs *all* parse trees from that table.

The *Grammar Editor* is text-centric, infers terminals/nonterminals from rules, lets users set the start symbol from the current rule, and supports configurable special symbols. Grammars persist as `.cgm` files (Python *shelve*); CNF conversion can be triggered from the menu. The *Parse Tree Editor* (FloatCanvas) has separate *view* and *edit* modes: zoom, expand/collapse subtrees, cycle through alternative subtrees when ambiguity is present, and save/load or export trees.

Using the standard "balanced parentheses" grammar $S \to (S) \mid SS \mid \epsilon$, CGM will both *accept* strings like `()(())` and, crucially for teaching, *display* distinct parse trees whenever the grammar is ambiguous (as it is here). Watching the editor step through CYK's table and then toggling the alternative subtrees makes the source of ambiguity visible instead of abstract.

3.3 ANTLR

ANTLR (ANother Tool for Language Recognition) is a production-grade parser generator used to build parsers, lexers, and tree walkers from declarative grammars. From a single `.g4` file, ANTLR emits target-language code and a parser that constructs parse trees, with optional *listener* and *visitor* APIs for deterministic tree processing. It is widely used to

implement domain-specific languages, query languages, configuration readers, and full compilers/interpreters in industry settings.

ANTLR4 supports multiple code generation targets so users can integrate generated parsers directly into their existing stacks. Official distributions and runtimes are available for each target, and IDE/build tooling (e.g., IntelliJ/Eclipse plugins, Maven/Gradle) makes it straightforward to incorporate into CI.23

In daily use, users (i) write a grammar, (ii) generate the lexer/parser, and (iii) attach behaviour by implementing either the *listener* pattern (event callbacks on enter/exit of rules via a tree walker) or the *visitor* pattern (explicit, return-value-oriented traversal).

3.4 YACC

Yacc (Yet Another Compiler-Compiler) is a classic Unix parser generator that takes a context-free grammar annotated with semantic and produces a bottom-up *LALR(1)* shift-reduce parser. In practice, Yacc is paired with a lexical analyzer, the scanner turns characters into tokens, and the generated parser organizes those tokens according to the grammar and executes user supplied actions to build trees, emit IR, or report errors.

Yacc's input file is divided into *declarations*, *grammar rules*, and *auxiliary C code*; tokens, precedence, and associativity are declared up front, grammar productions come with inline C actions using the familiar `$$/$i` value notations, and helper functions live at the end. The generated parser is a C function (`yyparse`) that repeatedly invokes the scanner (`yylex`) and applies reductions; common operator ambiguities are resolved declaratively with `%left`, `%right`, `%nonassoc`, and per-rule `%prec` to assign precedence and associativity.

This tool is well-suited for introducing *syntax analysis* and the mechanics of LR parsing while keeping implementation effort manageable: users define tokens in *lex/flex*, write a small expression or statement grammar in Yacc, attach actions to construct AST nodes or evaluate expressions, and then compile and run a working parser (e.g., a four-function calculator or a tiny language front end). Numerous tutorials and course handouts walk through this lex-yacc workflow step by step, making it an enduring choice in compiler courses.

4

OCamlFLAT/OFLAT

4.1 OCamlFLAT

The OCamlFLAT library is organized into modules to ensure a clearer and more structured organization. There are modules that implement models of FLAT theory, such as regular expressions, finite automata, and context-free grammars, as well as auxiliary modules for tests, error handling, and parsers. It is important to note that the modules follow a hierarchical relationship among themselves, which facilitates the maintenance and expansion of the library.

Some of the most important modules are:

Entity: This module represents the most abstract level of the library. The entities are the exercises and the FLAT models, each containing a type, a description, and a unique ID.

Exercise: This module is responsible for supporting exercises, which are validated using UnitTests. It extends the Entity module, inheriting its properties and methods.

Model: An abstract module that also extends Entity and introduces common functionalities to the FLAT models available in the library. Examples of methods included in this

module are validate, which checks the validity of a model, and accept, which processes inputs according to the model.

Examples of modules that represent FLAT models and extend Model include:

RegularExpression: This module contains all the functionalities needed to work with regular expressions, allowing the definition and manipulation of these expressions.

FiniteAutomaton: This module implements the functionalities of a finite automaton, including the definition of states and transitions.

ContextFreeGrammar: This module offers the functionalities of a context-free grammar, allowing the definition of production rules and the analysis of strings.

PushdownAutomaton: This module contains the functionalities of a pushdown automaton, which is an extension of finite automata with an additional stack for temporary storage of information.

4.2 OFLAT Overview

This chapter introduces the OFLAT/OCamlFLAT system in its current state. The system comprises two main components: the OFLAT web application, which provides a graphical user interface, and OCamlFLAT, the underlying library that handles the system's logic, including all FLAT models and their representations.

OFLAT is an ongoing development at the NOVA-LINCS research center within the Computer Science Department of FCT-UNL, at the beginning was supported by two projects: Factor (Functional ApproaCh Teaching pOrtuguese couRses) and LEAFs [8]. It is implemented in OCaml, utilizing the `js_of_ocaml` and Cytoscape.js libraries. Additionally, it leverages the OCamlFLAT library for logical operations. A screenshot of the OFLAT platform's user interface is shown in Figure 4.1

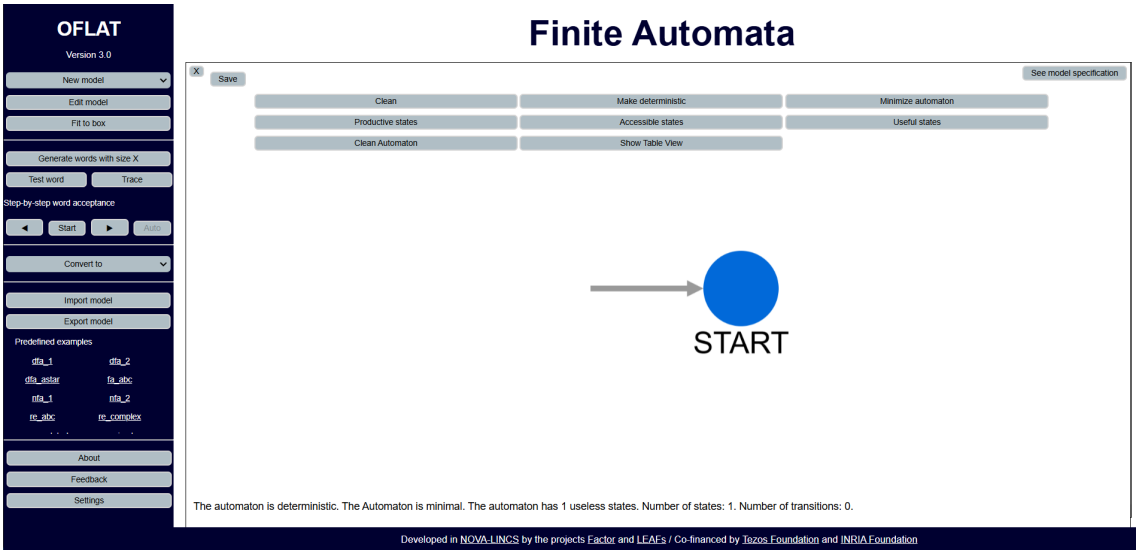


Figure 4.1: Current Version OFLAT User Interface

It supports working with a wide range of previously introduced language models, covering all types defined by Chomsky. This tool serves as a valuable resource for students as a study aid and for educators as a teaching tool worldwide. Its intuitive graphical interface distinguishes it from most other tools discussed in this document, many of which are outdated and lack modern or functional graphical interfaces.

This section provides a detailed examination of the internal workings of OFLAT and the libraries it relies on.

4.3 OFLAT User Interface

The OFLAT User Interface (UI) is primarily divided into two main sections: the workspace and the sidebar (as shown in Figure 4.2).

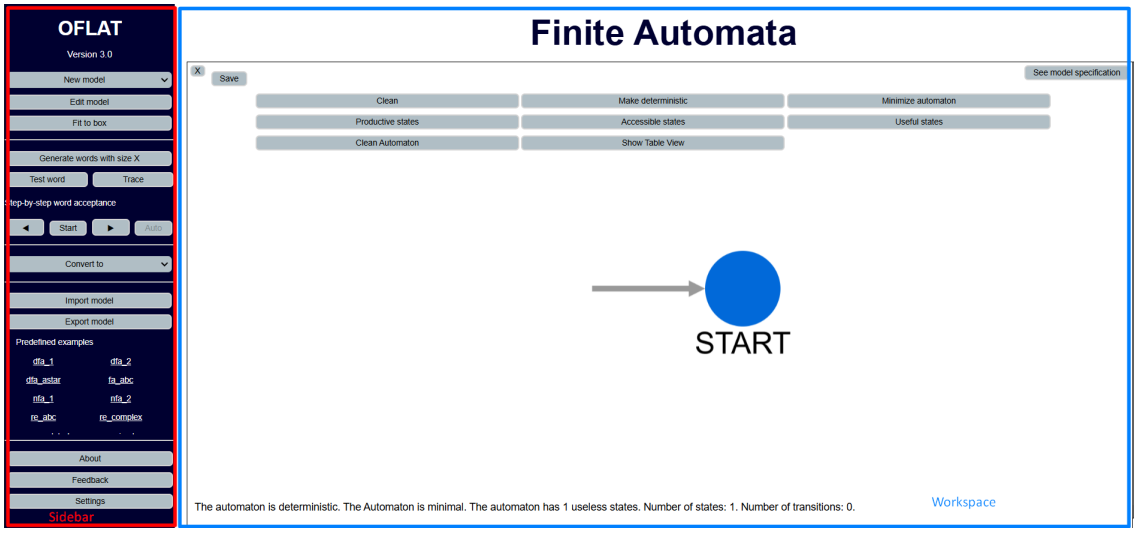


Figure 4.2: OFLAT User Interface Division. Red - Sidebar, Blue - Workspace

The sidebar contains buttons for creating and editing models within the workspace. Additionally, it provides access to key operations such as conversions and word acceptance. Alongside the About, Feedback, Language, and Settings buttons, there is also an Examples tab, which features a scrollable list of exercises and examples curated by the OFLAT development team. These examples allow users to test their knowledge or learn through guided practice.

The workspace section serves as the main area where users can visualize and interact with their chosen models. Depending on the selected model, a set of relevant buttons appears at the top of the workspace, offering access to specific operations. The workspace layout is illustrated in Figure 4.3.

However, when displaying the result of an operation or visualizing an exercise, the workspace splits into two sections. The original remains on the left side, while a window opens on the right, displaying the specification or exercise, creating the previously mentioned split (as shown in Figure 4.4). This layout allows users to view both elements simultaneously, making it easier to correlate them with minimal effort.

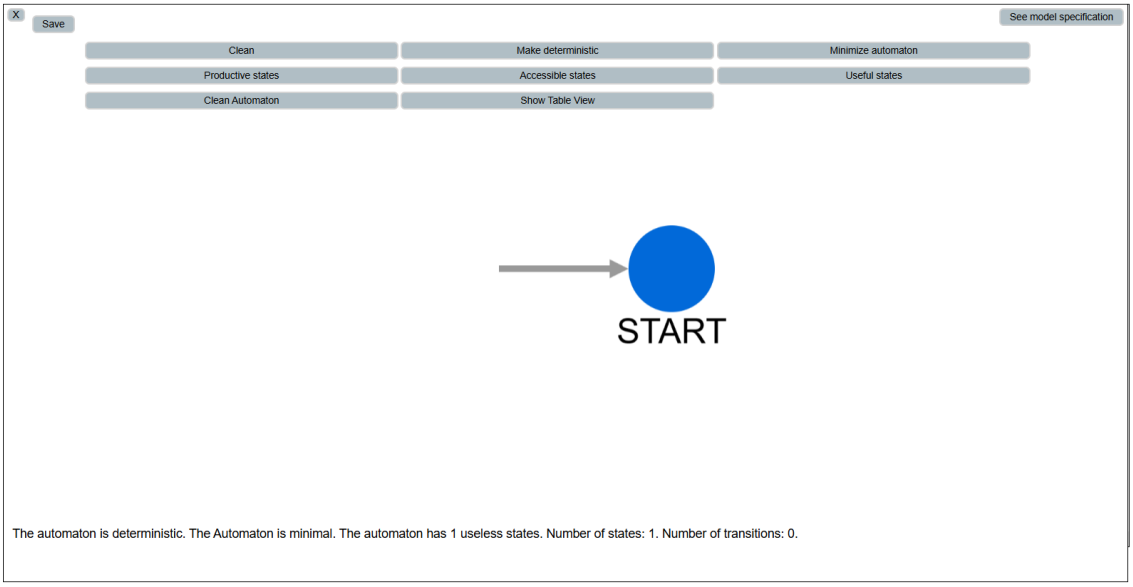


Figure 4.3: Workspace Closeup

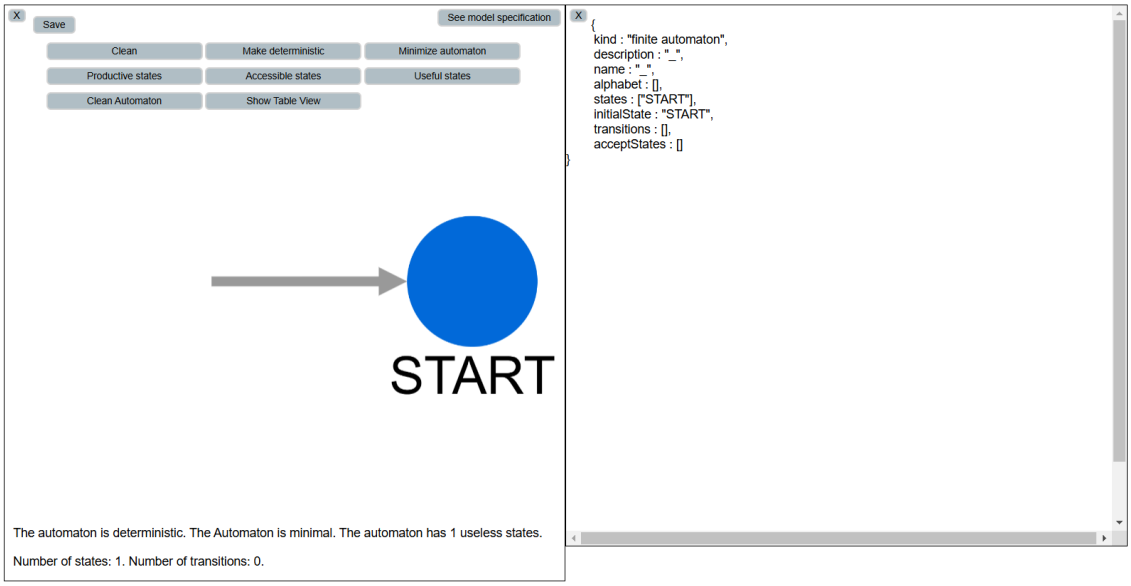


Figure 4.4: Workspace Window Split

5

Technology

5.1 Programming with OCaml

OCaml is a high-performance, portable functional programming language developed in 1996 by Xavier Leroy, Jérôme Vouillon, Damien Doligez, and Didier Rémy at INRIA in France [7]. Like other functional programming languages, OCaml is built around the concept of expressions, meaning computations are performed by evaluating expressions, which in turn produce values.

The example below, shows a few examples of OCaml code:

```
(* Definition of the binary tree type *)
# type tree = Node of int * tree * tree | Nil;;
- : type tree = Node of int * tree * tree | Nil


(* Definition of a variable that represents a tree *)
# let t : tree = Node(1, Nil, Node(2, Nil, Nil));;
- : val t : tree = Node(1, Nil, Node(2, Nil, Nil));;

(* Function that calculates the height of a tree *)
# let rec height t =
  match t with
  | Nil -> 0
  | Node(_,l,r) ->
         1 + max (height l) (height r)
;;
- : val height : 'tree -> int *

(* Calling the fuction *)
# height t;;
- : int = 2
```

5.1.1 Declarative Programming

In functional programming languages like OCaml, the code is often written in a declarative style. Declarative programming focuses on describing the logic of a computation rather than controlling its flow, this approach offers several advantages, for example by abstracting away the flow of control, the code becomes simpler compared to imperative programming. Additionally, since the code aligns more closely with the underlying concepts of the project, it tends to be more understandable and less prone to logical errors. Although this may come at the cost of efficiency, it benefits the OFLAT platform and OCamlFlat library, where the theoretical concepts involved are complex.

5.2 OCaml/JavaScript Interoperability

Interoperability between OCaml and JavaScript is made possible through the use of the `js_of_ocaml` toll [3] , which translates OCaml code into JavaScript, allowing OCaml code to run in a browser.

This tool consists of three main components:

- `js_of_ocaml` ppx: A preprocessor that extends the OCaml library, adding functionalities that facilitate integration with JavaScript.
- `js_of_ocaml` library: A library that contains bindings for various categories of JavaScript objects, enabling OCaml code to efficiently interact with JavaScript APIs and libraries.
- `js_of_ocaml` compiler: A compiler that translates OCaml code into JavaScript, allowing OCaml programs to be executed in web-based environments.

5.2.1 JavaScript Types

Types in OCaml and JavaScript are not represented in the same way in both languages. For example, a string in OCaml is a mutable array of bytes, whereas in JavaScript, strings are constant arrays of UTF-16 code points. Therefore, it is necessary to find a way to represent JavaScript types in OCaml appropriately. The table below illustrates the correspondences between the types in the two languages.

To ensure effective interoperability, it is crucial to understand how each type is handled in OCaml and JavaScript. For instance, besides strings, other types such as numbers, arrays, and objects also have distinct representations. In OCaml, integers and floating-point numbers are treated differently compared to JavaScript, where all numbers are essentially floating-point. Arrays in OCaml are mutable, while in JavaScript, arrays can be either mutable or immutable, depending on the context.

5.2.2 Bindings

Bindings are present in both the libraries within the *js_of_ocaml* environment and the OFLAT code itself. They allow the connection between two different programming languages, enabling the use of a library foreign to one of them.

Primarily, *js_of_ocaml* manages bindings by assigning JavaScript objects a distinct OCaml object type. This type is defined as `<m1 : t1; ...; mn : tn> Js.t`, where m_i represents a field of the JavaScript object and t_i its corresponding type.

5.2.3 `js_of_ocaml` Modules

The interoperability between OCaml and JavaScript is handled by the *js_of_ocaml* compiler in different ways. Within the compiler, the `Js` module exists to manipulate JavaScript values (such as strings, booleans, null, and undefined values) from OCaml and to perform conversions between the two languages. For example, the type `'a Js.opt` represents values that are either `null` or of type `'a`.

Additionally, this module includes another important module: `Js.Unsafe`. It provides a collection of unsafe JavaScript operations, ranging from type coercion to setting, getting, and deleting object properties. These operations are very powerful and should ideally be used sparingly across the project. However, in some cases, their use may be necessary. By creating bindings and ensuring proper typing, most of these calls can be avoided.

The `Js` module is one of several modules included in the libraries that come with the compiler. Other key modules worth mentioning are `Dom`, which serves as a partial binding to the DOM Core API; `Dom_html`, which acts as a partial binding to the DOM HTML API; and the `Firebug` module, which provides a debugging console for development.

5.3 Cytoscape.js

Cytoscape.js is an open-source graph theory library written in JavaScript that enables the manipulation of graphs. It functions as a standalone JavaScript library and utilizes HTML5 canvases for rendering its visualization components.

In OFLAT, Cytoscape.js is primarily used for the graphical representation of automata through nodes and edges. Additionally, it is notably used whenever OFLAT needs to display a tree (important in the scope of context-free grammars).

With this library, users can perform various operations on the displayed automaton, such as creating new states and transitions, allowing for interactive manipulation of the model.

6

OCAML-FLAT - Implementation

OCaml-FLAT is one of the two main components that are part of the existing structure of this project. It consists of the core logic of the tool and is implemented entirely in OCaml. In this chapter, we will address OCaml-FLAT, which contains the main operations implemented for attribute grammars.

OCaml-FLAT is a library composed of several modules, and each module is built as a class. This library, up to the moment of writing this document, implements pushdown automata, regular expressions, context-free grammars, as well as some supporting modules, such as operations on JSON files, errors, parsers, among others.

For the implementation of attribute grammars in OCaml-FLAT, the central goal is to couple a precise static validation phase with a deterministic evaluation strategy over parse trees. The library represents an attribute grammar as a set of rules with equations and conditions attached to each production, and it enforces a good construction before any evaluation can proceed. Validation checks range from context-free sanity (reusing CFG validation) to attribute typing, direction constraints (inherited vs. synthesized), and global acyclicity of attribute dependencies. Only after this pipeline succeeds do we execute the attribute equations over a parse tree.

Two aspects are key to the design. First, expression typing and equation validation ensure each assignment is meaningful and referentially correct. Second, the evaluation order guarantees that inherited attributes of a child are computed before descending into that child, and synthesized attributes of the head are computed after its children return—yielding a left-to-right, node-local schedule that is predictable and terminating in the absence of dependency cycles. Together, these choices produce a robust and explainable evaluator without relying on heuristic fixpoint closures.

Typing proceeds by a straightforward rule: attributes are associated to a base type inferred from their name—by default integers, with `"s"` denoting string and `"b"` denoting boolean—while constants and operators are type checked accordingly. Binary operators are admitted only when operand types align (e.g., `+` on `int` or `string`, `-`, `*`, `/` on `int`, comparisons across equal types), and any mismatch is flagged at validation time. Applications of attributes use `(attr, (var, i))`, where `i` identifies the head or a specific

child occurrence; an index normalization step resolves the default case and unambiguous references.

On the left-hand side of equations, the direction discipline is enforced: assignments targeting the head must write synthesized attributes, whereas assignments targeting a child must write inherited attributes. Indices are checked against the rule's body to prevent out-of-range references, and all variables must belong to the grammar's variable set. If types on both sides of an equation do not match, validation fails. Conditions are validated separately and must type check to boolean.

At evaluation time, the library models a local environment as a list of nodes, headed by the current symbol and followed by its children. Each node carries the set of evaluated attributes for that symbol occurrence. Attribute applications resolve by locating the appropriate node—head or child occurrence—and reading the requested attribute value. Updates are performed by replacing the targeted attribute binding on the designated node, preserving immutability of the structure. A stable ordering of equations at the head and per child ensures determinism of side-effect-free updates within a node.

The evaluation strategy is left-to-right on children. For each child, all inherited equations targeting that child are applied in the head's environment before the recursive descent. The child is then evaluated. Once all children have been processed, equations that target the head (i.e., synthesized attributes) are evaluated, and node-local conditions are checked. This yields a two-phase per-node schedule—child inherited first, head synthesized later—that matches the intuition for attribute flow while avoiding ad hoc closures over ε-like effects.

Global correctness is guarded by a dependency graph construction. For every equation, the implementation identifies the destination attribute occurrence (head or specific child) and the set of attribute references appearing on the right-hand side. From these, it builds a directed graph whose nodes are "(symbol, occurrence, attribute)" triplets and whose edges represent "RHS depends on LHS." A depth-first search detects cycles; if any cycle exists, the grammar is rejected as having circular attribute dependencies. This check is integrated into the overall `validate` routine, preventing non-terminating evaluations at runtime.

The attribute grammar layer interacts with the context-free layer. Conversions are provided in both directions: an attribute grammar can be projected to a CFG by erasing attributes, and a CFG can be lifted to an attribute grammar with empty attribute sets and equation blocks. Word acceptance may, when appropriate, delegate to the CFG acceptance procedure, while a separate routine attempts attribute evaluation on a given parse tree, reporting success or failure. A generator uses the CFG projection to enumerate terminal strings up to depth and count limits, which is useful for testing attribute specifications independently of a parser.

Reference semantics for attribute occurrences are explicit. Index `0` denotes the head; index `-1` is normalized to the head when the variable equals the head, and otherwise to the first occurrence of the named child; positive indices select the k-th matching child.

Occurrence computations and equation ordering are derived from the rule body, ensuring that references are stable and reproducible even when the same child symbol appears multiple times. This removes ambiguity at both validation and evaluation time.

6.1 Data types created for attribute grammars in Ocaml

The main data types defined to simplify the manipulation of attribute grammars are: `t`, which represents an attribute grammar (alphabet, variables, synthesized and inherited attributes, initial variable, and set of rules); `rule`, which models a production with *head*, emphbody, a set of `equations`, and a set of `conditions`; and `equation`, which represents an assignment between `expressions`. Additionally, `expression` and `value` describe the expressions and constants used in equations; `condition` represents boolean predicates; and `attribute`, `attributes`, and `attrArg` handle the identification of attributes and their occurrences.

```
type attribute = symbol
    type attributes = attribute set
    type attrArg = variable * int
    type value =
        | Int of int
        | String of string
        | Bool of bool
    type expression =
        | Const of value
        | Apply of attribute * attrArg (* l(A2) *)
        | Expr of string * expression * expression
    type equation = expression * expression
    type equations = equation set
    type condition = expression
    type conditions = condition set

    type rule = {
        head : variable;
        body : word;
        equations : equations;
        conditions : conditions
    }
    type rules = rule set

    type t = {
        alphabet : symbols;
```

```
        variables : variables;
        synthesized : attributes;
        inherited : attributes;
        initial : variable;
        rules : rules
    }
```

6.2 The calcAttributes fuction

calcAttributes evaluates all attributes in a parse tree according to a given attribute grammar. It ensures that each child node receives its inherited attributes before that child is evaluated, and that the parent node computes its synthesized attributes only after all children have been evaluated. The function returns a tree with the same shape, but with attribute maps updated on the head node and on every descendant.

```
let rec calcAttributes (ag : t) (pt : parseTree) : parseTree =
    match pt with
    | Leaf _ -> pt
    | Node (((head_sym, _) as head), children) ->
        let rule = getRootRule ag (Node (head, children)) in
        let children' = eval_children_left_to_right ag head rule.equations children in
        let head_eqs = set_filter (eq_targets_head head_sym) rule.equations in
        let env2 = head :: List.map getRoot children' in
        let env3 = apply_equations_at_head head_sym env2 head_eqs in
        check_conditions_at_node head_sym rule.conditions env3;

        match env3 with
        | (new_head_sym, new_head_evs) :: child_envs ->
            let children'' = zip_update_children children' child_envs in
            Node ((new_head_sym, new_head_evs), children'')
        | [] ->
            failwith "calcAttributes: empty environment after synthesized equations"
```

6.2.1 Inputs and Output

The function takes an attribute grammar ag and a parse tree pt. If pt is a leaf, it is returned unchanged. If pt is an internal node, the function computes attributes for the node and for all its children, returning a structurally identical tree whose nodes carry the evaluated attribute bindings.

6.2.2 Evaluation Procedure at an Internal Node

Let the current node be `Node((head_sym, head_evs), children)`.

1. **Rule identification.** The function determines the unique grammar rule whose head symbol equals `head_sym` and whose body matches the sequence of child symbols at this node. If no rule matches, a diagnostic is raised that includes the encountered head/body and the candidate rules that share the same head. This step fixes the set of equations and conditions that apply at the node.

2. **Inherited attributes for children (left-to-right).** Children are processed strictly from left to right. For the current child occurrence, the function builds an *environment* that consists of the head node, the already processed children (in order, with any attributes computed so far), and the remaining children (not yet processed). From the node's rule, it selects only those equations that assign *inherited* attributes to this specific child occurrence (identified by symbol and 1-based occurrence index), and applies them to the environment in a stable order. The child's root is updated with these inherited values, and then `calcAttributes` is called recursively on that child to evaluate its subtree. This guarantees that a child's inherited attributes may depend on the parent and on earlier siblings, but not on later siblings.

3. **Synthesized attributes for the head.** Once all children are fully evaluated, the function gathers only the equations that target the head (i.e., assign *synthesized* attributes). It constructs a head-first environment containing the head node followed by the now-evaluated child roots and applies those equations in a deterministic, stable order, updating the head's attribute bindings.

4. **Condition checking.** The boolean conditions attached to the rule are evaluated in the same head-first environment. Each condition must evaluate to `true`. If a condition fails or evaluates to a non-boolean value, evaluation aborts with an error message that identifies the node at which it failed.

5. **Reconstruction.** Finally, the function rebuilds and returns the node by replacing the head and each child root with their updated attribute environments. The tree's topology is unchanged; only attribute maps are updated.

6.2.3 Deterministic Order and Rationale

The strict left-to-right order establishes a predictable flow of information: inherited attributes for child i may depend on the head and on children $1..i-1$, but never on $i+1..n$. This sequencing avoids evaluation-time circularities and ensures each child observes a stable context when it is evaluated. A separate dependency-graph check elsewhere detects global cycles among equations.

6.2.4 Error Handling

The surrounding code validates that attribute references target valid head/child positions, checks expression types (e.g., integer vs. string arithmetic and comparisons), and detects inconsistent environments (e.g., out-of-range child occurrence, head-symbol mismatches). Failures are reported with precise diagnostics at the point of detection; rule-selection mismatches list candidate rules with the same head, and condition failures indicate the node where the evaluation failed.

6.3 Auxiliary Methods Used by `calcAttributes`

This section focuses on the local worker that drives left-to-right child evaluation for `calcAttributes`. In the implementation, `calcAttributes` delegates the child pass to `eval_children_left_to_right`, which defines a local recursive function `loop`. This `loop` is the auxiliary routine that realizes the stepwise propagation of inherited attributes to each child and then triggers recursive evaluation of that child.

6.3.1 `eval_children_left_to_right`

Evaluate children in strict left-to-right order, applying the inherited-attribute equations that target each child occurrence before recursively evaluating that child's subtree. This ensures that each child receives its required inherited attributes possibly dependent on the head and on previously evaluated siblings, but not on later siblings.

This fuction receives a grammar `ag`, the current node's head `head`, the full set of equations for the node's rule `eqs`, and the list of child parse trees `children`. And returns a list of fully evaluated child parse trees, in original order.

The function computes the (symbol, occurrence) pair for each child (e.g., second `Expr` child). It defines a local recursive worker `loop` that maintains three lists: already processed children (reversed), remaining children, and the corresponding (symbol, occurrence) pairs for the remaining ones. For the current child, an evaluation environment is built as `head :: processed_roots @ remaining_roots`. Inherited equations targeting exactly this child occurrence are filtered and applied to the environment, producing the child's inherited attributes. The child root is updated with those attributes, the child is recursively evaluated via `calcAttributes`, and the loop advances to the next child. Any mismatch between children and occurrence annotations raises a clear failure.

```
and eval_children_left_to_right
    (ag : t)
    (head : node)
    (eqs : equation Set.t)
    (children : parseTree list)
    : parseTree list =
```

```
let head_sym = fst head in
let occs = child_occurrences children in
let rec loop
    (processed : parseTree list)        (* evaluated children, reversed *)
    (remaining : parseTree list)        (* children yet to process     *)
    (remaining_o : (symbol * int) list) (* their (sym,occ)             *)
    : parseTree list =
  match remaining, remaining_o with
  | [], [] ->
      List.rev processed
  | child :: rest, occ :: occs_rest ->
      let processed_nodes = List.rev processed |> List.map getRoot in
      let env_before =
        head :: (processed_nodes @ (List.map getRoot (child :: rest)))
      in
      let inh_eqs = set_filter (eq_targets_child head_sym occ) eqs in
      let env_after_inh =
        apply_equations_at_head head_sym env_before inh_eqs
      in
      let child_node_after_inh =
        match env_after_inh with
        | _head_after :: env_tail ->
            pick_child_node_by_occ occ env_tail
        | [] ->
            failwith
              "calcAttributes: empty environment after inherited equations"
      in
      let child_with_inh = updateRoot child child_node_after_inh in
      let child_evaluated = calcAttributes ag child_with_inh in
      loop (child_evaluated :: processed) rest occs_rest
  | _ ->
      failwith
        "calcAttributes: internal error (children/occurrences mismatch)"
in
loop [] children occs
```

The call to `eval_children_left_to_right` is the point where the local auxiliary function `loop` is used.

The `loop` function above is the auxiliary worker for `calcAttributes`.

6.3.2 Role of the Auxiliary `loop` Function

The `loop` fuction performs a left-to-right sweep over the node's children. For each child in order, it computes and writes that child's inherited attributes, recursively evaluates the child subtree, and carries the result forward before proceeding to the next child. This realizes the required dependency discipline: a child may depend on the head and on previously evaluated siblings, but not on later siblings.

The function carries three accumulators:

- `processed`: the list of children already evaluated; kept in reverse order for efficient cons operations. These are fully evaluated subtrees.
- `remaining`: the list of children yet to process, with the next child at the head.
- `remaining_o`: the list of $(symbol, occurrence)$ pairs corresponding one-to-one with `remaining`, used to target inherited-equation assignments to the correct child occurrence.

Key invariants

1. `processed @ remaining` is a permutation of the original `children`, preserving relative order between the still-unseen suffix and the already-evaluated prefix. The invariant is maintained by consing the newly evaluated child onto `processed` and dropping it from `remaining`.
2. `remaining_o` matches `remaining` position-by-position; each pair (s, k) gives the symbol and k-th occurrence index of the corresponding child among all children of this node. This allows selection of inherited equations and of the correct child node in the environment.
3. Every tree in `processed` is fully evaluated (its inherited attributes were set before the recursive call, and its synthesized attributes and conditions were handled inside that recursive `calcAttributes`).

6.3.3 Step-by-Step Operation of `loop`

For the case `child :: rest, occ :: occs_rest`:

1. Compute `processed_nodes` as the roots of already-evaluated children (in original order). Then build the evaluation environment for this step:

   `env_before` = `head` :: `processed_nodes` @ `(roots of (child :: rest))`.

   This environment exposes the head, earlier siblings, the current child, and later siblings to the equations to be applied.

2. Select only inherited-attribute equations that target the current occurrence `occ`:

   $$\texttt{inh_eqs} = \{\, e \in \texttt{eqs} \mid \texttt{eq_targets_child head_sym occ}(e) \,\}.$$

   Apply them in a stable order at the head of `env_before`:

   $$\texttt{env_after_inh} = \texttt{apply_equations_at_head head_sym env_before inh_eqs}.$$

   The effect is to update exactly the target child's inherited attributes in the environment.

3. Extract the updated root node for the current child occurrence from the environment tail using `pick_child_node_by_occ occ`, and write it back into the child tree with `updateRoot`:

   $$\texttt{child_with_inh} = \texttt{updateRoot child child_node_after_inh}.$$

   If the environment is malformed or the occurrence is not found, a descriptive failure is raised.

4. Recursively evaluate the child subtree:

   $$\texttt{child_evaluated} = \texttt{calcAttributes ag child_with_inh}.$$

   This computes the child's own synthesized attributes and checks its local conditions.

5. Advance the loop:

   $$\texttt{loop (child_evaluated :: processed) rest occs_rest}.$$

   The measure |`remaining`| strictly decreases, ensuring termination.

When `remaining` and `remaining_o` are both empty, the loop returns `List.rev processed`, i.e., the fully evaluated children in original left-to-right order. Any other shape indicates a mismatch between children and occurrence annotations and triggers an explicit failure.

6.3.4 The Loop Necessity

The local `loop` encodes the evaluation discipline that inherited attributes of child i may depend on the head and on the already evaluated siblings $1..i-1$, but not on later siblings. It does so by constructing `env_before` at each step, selecting inherited equations for exactly one occurrence, and committing the results before recursing to the next child. This realizes a deterministic, left-to-right strategy and prevents accidental dependence on future siblings during inherited-attribute computation.

6.3.5 Relation Back to `calcAttributes`

After `loop` has produced the fully evaluated children, `calcAttributes` proceeds to the parent's synthesized pass and condition checking. The correctness of those later phases relies on the contract guaranteed by `loop`: all children are available with their inherited attributes applied and their subtrees fully evaluated.

6.3.6 `getRootRule`

Select the unique grammar rule that matches the current parse-tree node: the rule whose head symbol equals the node's root symbol and whose body (sequence of symbols) equals the sequence of the node's immediate children. If the node is a leaf, there is no applicable rule and the function fails. If no rule matches, the error message lists candidate rules that share the same head symbol to aid diagnosis.

Input: the attribute grammar `ag` and a parse tree `pt`. Output: the matching rule `rule`. Failure cases: leaf nodes, or internal nodes whose children do not match any rule body for the node's head symbol.

For an internal node, the head symbol is taken from the node's root and the body is the list of child-root symbols. The function searches `ag.rules` for a rule whose head and body match. If none matches, it constructs a readable error with candidate alternatives.

```
let getRootRule (ag: t) (pt: parseTree): rule =
  match pt with
  | Leaf _ ->
      failwith "getRootRule: leaf has no rule"
  | Node (_, children) ->
      let head = getRootSymbol pt in
      let body = List.map getRootSymbol children in
      try Set.find (fun r -> r.head = head && r.body = body) ag.rules with
      | _ ->
          let candidates =
            ag.rules
            |> SetUtil.to_list
            |> List.filter (fun r -> r.head = head)
            |> List.map (fun r ->
                 Printf.sprintf "- %s -> %s"
                   (symb2str r.head)
                   (String.concat " " (List.map symb2str r.body)))
            |> String.concat "\n"
          in
          failwith (Printf.sprintf
            "getRootRule: no rule matches head=%s, body=[%s]\nCandidates with same head:\
```

```
            (symb2str head)
            (String.concat " " (List.map symb2str body))
            (if candidates = "" then "(none)" else candidates))
```

6.3.7 apply_equations_at_head

Apply a given set of equations to an evaluation environment whose first node must be the head node with a specific symbol. Equations are applied in a stable, deterministic order to avoid order-dependent results. The output is a new environment reflecting the updates introduced by these equations (e.g., inherited assignments to a specific child, or synthesized assignments to the head).

Inputs: head symbol `head_sym`, a non-empty environment `nodes` whose first node has symbol `head_sym`, and an equation set `equations`. Output: an updated environment `node list`. Failure cases: empty environment or a head-symbol mismatch at the first node.

The function first verifies that the environment is not empty and that its first node matches `head_sym`. It then orders the equations using a stable ordering and folds over them, applying each equation with `eval` to produce the next environment. The fold result is the updated environment.

```
let apply_equations_at_head
    (head_sym : symbol)
    (nodes : node list)
    (equations : equation Set.t)
    : node list =
  let () =
    match nodes with
    | [] ->
        failwith "apply_equations_at_head: empty environment (nodes)"
    | (hs, _) :: _ when hs = head_sym -> ()
    | (hs, _) :: _ ->
        failwith
          (Printf.sprintf
             "apply_equations_at_head: head symbol mismatch (expected %s, got %s)"
             (symb2str head_sym) (symb2str hs))
  in
  let eqs = stable_eq_order head_sym equations in
  List.fold_left
    (fun acc_nodes eq -> eval head_sym eq acc_nodes)
    nodes
    eqs
```

6.3.8 check_conditions_at_node

Evaluate the rule's boolean conditions at the current node's environment and ensure they all hold. A failing condition aborts evaluation with a message identifying the node's head symbol.

Inputs: head symbol `head_sym`, the set of conditions `conds`, and the current environment `env`. Output: `unit`. Failure cases: any condition evaluates to a non-boolean value or to `false`.

Each condition is evaluated with `evaluate`; the result is coerced with `as_bool`. If any evaluation fails to yield `true`, a failure is raised with a diagnostic mentioning the head symbol.

```
let check_conditions_at_node
    (head_sym : symbol)
    (conds : condition Set.t)
    (env : node list)
    : unit =
  Set.iter (fun cond ->
    let v = evaluate head_sym cond env in
    if not (as_bool v) then
      failwith (Printf.sprintf
        "Condition failed at node %s"
        (symb2str head_sym))
  ) conds
```

6.4 The accept fuction

The `accept` receives a word and an attribute grammar, and returns a boolean indicating whether the word is accepted by the grammar. The function first converts the attribute grammar into an equivalent context–free grammar and then relies on the acceptance mechanism of that grammar.

```
let accept (ag: t) (w: word): bool =
    let cfg = ga_to_cfg ag in
    ContextFreeGrammarBasic.accept cfg w
```

6.5 The validate function

The `validate` performs the static checks required before evaluating an attribute grammar on any parse tree. It combines five steps:

1. Convert the attribute grammar to a plain context-free grammar (CFG),

2. Validate the CFG structure,
3. Type and well-formedness-check all attribute equations,
4. Ensure all rule conditions are boolean,
5. Detect dependency cycles among attribute computations.

If any check fails, an error is raised with an explanatory message.

The fuction receives a user-facing name `name` (used in diagnostics) and an attribute-grammar value `rep`. And returns normally if and only if all checks pass. Otherwise it reports an error with `Error.error` or raises a failure from helper routines.

The grammar is first projected to a CFG with `ga_to_cfg`. This preserves terminals, variables, initial symbol, and the bare head→body production structure. The resulting CFG is then validated by `ContextFreeGrammarPrivate.validate`, which checks basic well-formedness (e.g., symbols belong to the declared sets, rules reference known symbols, etc.).

`validateEquations` traverses all rules and their equations. For each equation `lhs = rhs`:

- the left-hand side must be an `Apply(attr,(var,i))` to a valid target (head or a concrete child occurrence in range); the variable must be a declared grammar variable; and the direction is checked: assignments to the head require synthesized attributes, and assignments to children require inherited attributes;
- both sides are type-checked via `validateExp`, ensuring equal, non-`"error"` types. Operators are checked for arity and operand types (e.g., integer arithmetic, string concatenation, comparisons).

Any mismatch (invalid target, out-of-range occurrence, wrong direction, unknown symbol/attribute, or type error) triggers an error at this stage.

`validateConditions` ensures every condition expression attached to a rule statically type-checks as boolean (again using `validateExp`). Non-boolean conditions are rejected with a diagnostic.

`has_cycles` constructs a directed dependency graph whose nodes are attribute occurrences identified by (*symbol*, *occurrence*, *attribute*). For each equation, edges are added from every attribute reference in the right-hand side to the left-hand side target, within the context of the same rule. A depth-first search detects any cycle; if present, validation fails with "Attribute dependency cycle detected". This prevents unschedulable attribute computations at evaluation time.

```
let validate (name: string) (rep: t): unit =
  let cfg = ga_to_cfg rep in
  ContextFreeGrammarPrivate.validate name cfg;
```

```
  validateEquations name rep;
  validateConditions name rep;
  if has_cycles rep then
    Error.error name "Attribute dependency cycle detected" ()
```

6.5.1 Auxiliary Methods Used by `validate`

The following helpers iterate over all rules and check each equation or condition individually:

```
let validateEquations (name: string) (rep: t): unit =
  Set.iter (fun r ->
    Set.iter (fun eq ->
      validateEquation name rep eq r
    ) r.equations
  ) rep.rules

let validateConditions (name: string) (rep: t): unit =
  Set.iter (fun r ->
    Set.iter (fun eq ->
      validateCondition name rep eq r
    ) r.conditions
  ) rep.rules
```

These use rule-scoped checkers: `validateEquation` confirms target well-formedness, direction (inherited vs. synthesized), variable membership, and type equality between left and right expressions; `validateCondition` verifies that the condition's static type is boolean.

The cycle check reduces to a graph problem:

```
let has_cycles (ag : t) : bool =
  ag |> build_dep_graph |> has_cycle_graph
```

Here `build_dep_graph` indexes each attribute occurrence into a node id and collects edges from RHS references to LHS targets for every equation of every rule. `has_cycle_graph` runs a DFS-based cycle test over the adjacency lists. If any back-edge is found, the function returns `true`.

6.6 Cycle Detection

Attribute dependencies are modelled as a directed graph: each node represents a concrete attribute occurrence $(symbol, occurrence, attribute)$, and each equation adds edges from every attribute referenced on the right-hand side to the left-hand side target. A DFS-based test on this graph reports whether any cycle exists. If a cycle is found, validation fails.

6.6.1 build_dep_graph

this fuction constructs the dependency graph of the whole grammar and return it as adjacency lists.

- Map each occurrence (s, k, a) to a unique node id (hash table).
- For each rule equation $LHS = RHS$: translate LHS to its destination node; collect all attribute refs in RHS and add edges ref $\rightarrow LHS$.
- Materialize the edge set into an `int list array`.

```
type node_key = symbol * int * attribute

let build_dep_graph (ag : t) : int list array =
  let index_tbl : (node_key, int) Hashtbl.t = Hashtbl.create 64 in
  let counter = ref 0 in
  let get_id key =
    match Hashtbl.find_opt index_tbl key with
    | Some id -> id
    | None ->
        let id = !counter in
        incr counter;
        Hashtbl.add index_tbl key id;
        id
  in
  let edges = ref [] in
  let rules = SetUtil.to_list ag.rules in
  List.iter (fun r ->
    Set.iter (fun eq ->
      match eq with
      | Apply (attrL, (vL,iL)), rhs ->
          let (sL,jL) = sym_occ_of_ref r (vL,iL) in
          let dst = get_id (sL, jL, attrL) in
          let refs = refs_in_expr r rhs in
          List.iter (fun (s,j,a) ->
            let src = get_id (s,j,a) in
            edges := (src, dst) :: !edges
          ) refs
      | _ -> ()
    ) r.equations
  ) rules;
  let n = !counter in
```

```
  let adj = Array.make n [] in
  List.iter (fun (u,v) -> adj.(u) <- v :: adj.(u)) !edges;
  adj
```

6.6.2 has_cycle_graph

This fuction decides if the directed graph contains any cycle using DFS with a recursion stack.

Maintain two boolean arrays: `visited` and `stack`. A back-edge to a node currently in `stack` signals a cycle; otherwise, explore all successors.

```
let rec dfs adj u visited stack =
  if stack.(u) then true
  else if visited.(u) then false
  else (
    visited.(u) <- true;
    stack.(u) <- true;
    let found = List.exists (fun v -> dfs adj v visited stack) adj.(u) in
    stack.(u) <- false;
    found
  )

let has_cycle_graph adj =
  let n = Array.length adj in
  let visited = Array.make n false in
  let stack = Array.make n false in
  let rec loop i =
    if i = n then false
    else if (not visited.(i)) && dfs adj i visited stack
    then true
    else loop (i+1)
  in
  loop 0
```

6.7 Unit Tests: Trees and Attribute Annotations

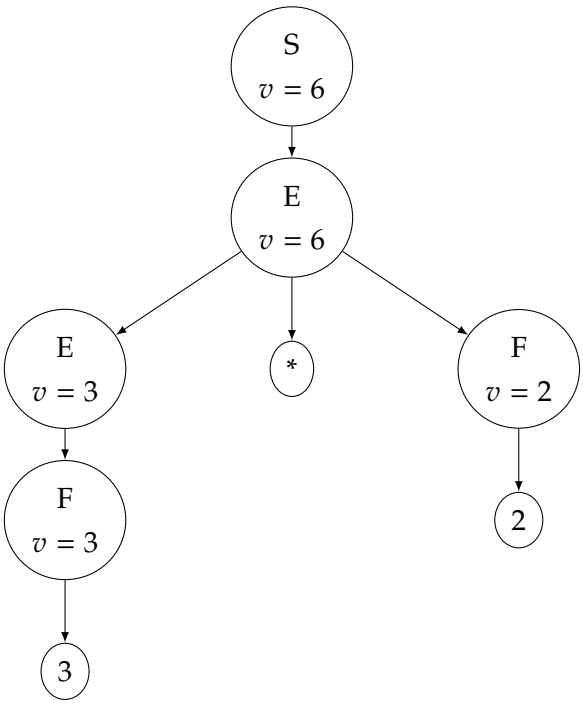
This chapter includes every unit test defined in the test module, with a brief purpose statement and a corresponding figure. Passing cases show fully annotated attribute values; failure-oriented cases indicate the precise failure point (type mismatch, division by zero, L-attributed violation, or out-of-range occurrence). Cycle-detection tests also include the attribute-dependency graph.

6.7.1 `test_ag1_simple_synthesized` (ag1, tree `pt1`)

Simple synthesized value over multiplication: $S \to E$, $E \to E * F$, inner $E \to F$; leaves $F \to 3, F \to 2$. Expected $v(S) = 6$.

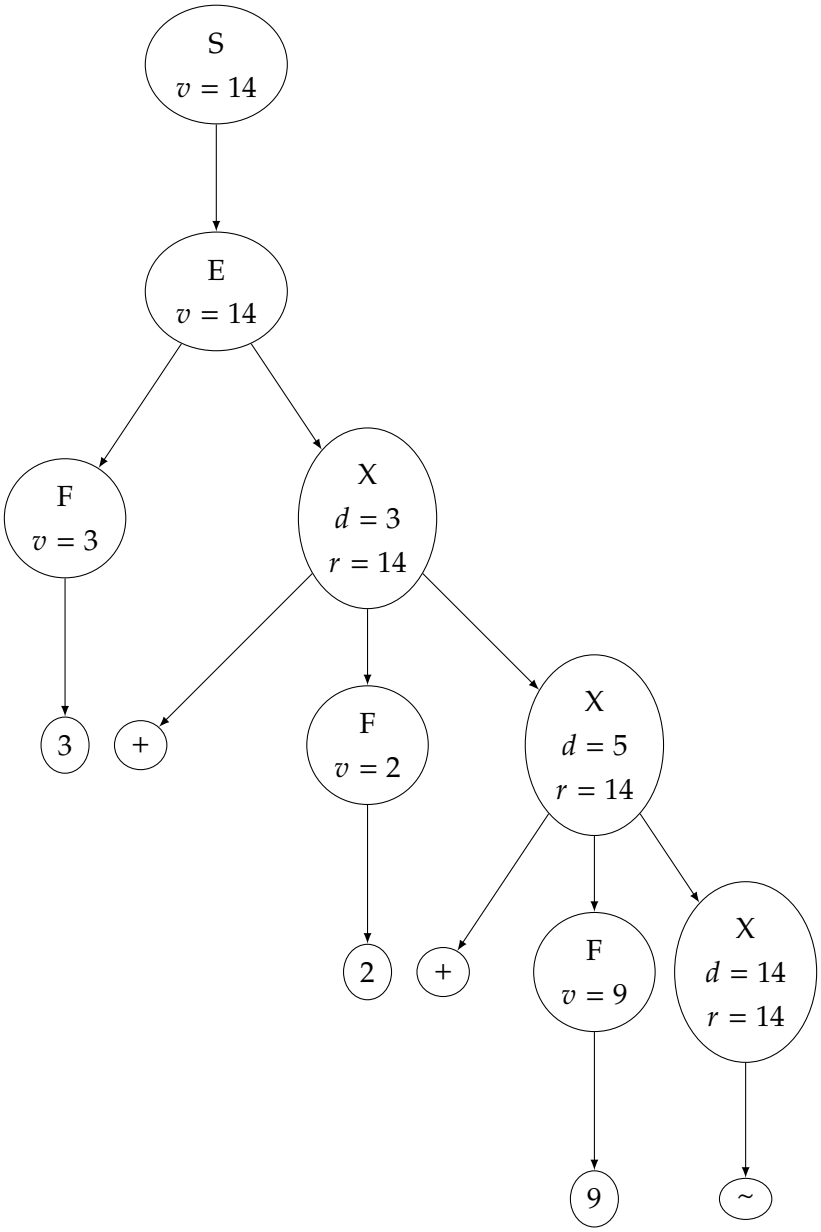
```
Synthesized: ["v"]

S -> E {v(S) = v(E)}
E -> E * F {v(E0) = v(E1) * v(F)}
E -> F {v(E) = v(F)}
F -> 2 {v(F) = 2}
F -> 3 {v(F) = 3}
```



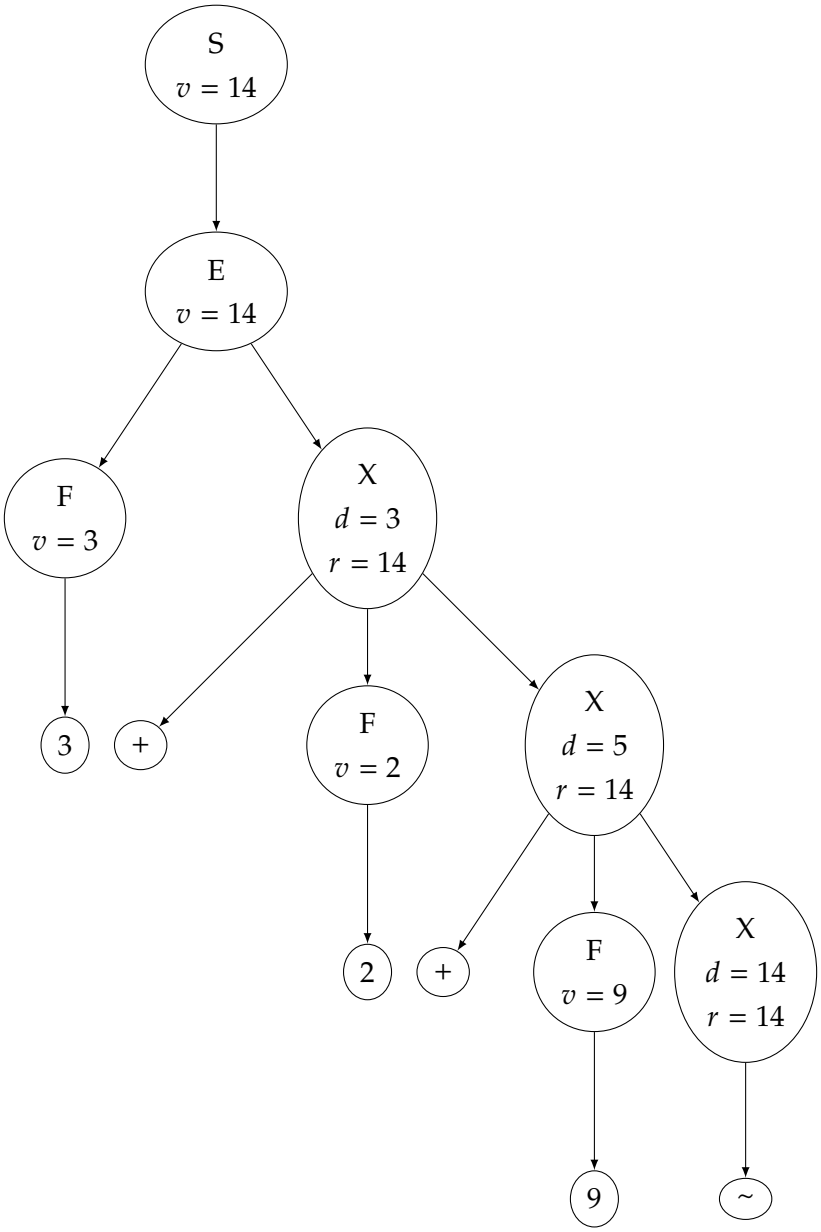
6.7.2 `test_parseTree` (Grammar `ag2`, word `3+2+9~`)

Parse `3+2+9~` from the CFG induced by `ag2`, convert to an AG tree, and evaluate attributes. Rules propagate a running sum via inherited `d` and synthesized `r`; result at root: $v(S) = 14$.



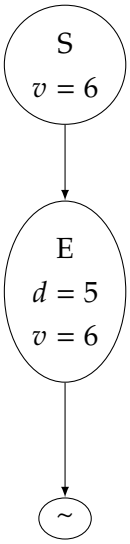
6.7.3 `test_ag2_simple_synthesized_and_inherited (ag2, tree pt2)`

Same structure as `test_parseTree`; checks inherited `d` and synthesized `r` along the `X`-chain; $v(S) = 14$.



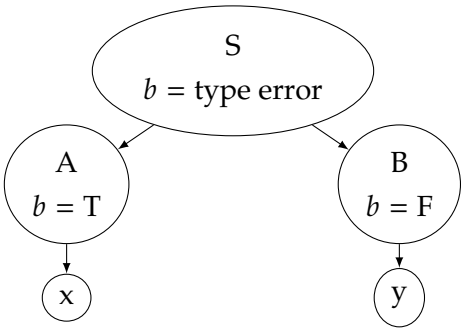
6.7.4 `test_ag3_simpler_synthesized_and_inherited` (ag3, tree `pt3`)

Minimal inherited/synthesized flow: $S \rightarrow E\ \{d(E) = 5,\ v(S) = v(E)\}$, $E \rightarrow\sim\ \{v(E) = d(E) + 1\}$. Expected $v(S) = 6$.



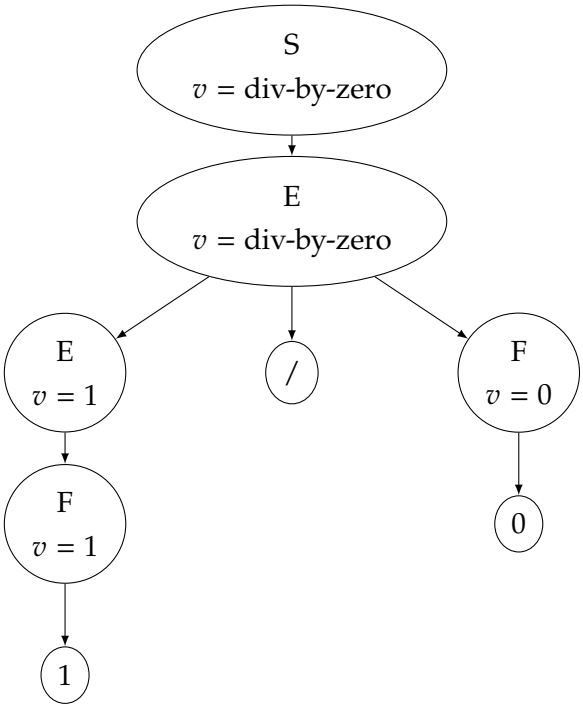
6.7.5 `test_ag4_type_mismatch` (ag4, tree `pt4`)

Deliberate type error: $S \to A\ B\ \{\, b(S) = b(A) + b(B) \,\}$ with $A \to x\{b(A) = \mathrm{T}\}$, $B \to y\{b(B) = \mathrm{F}\}$. The sum + is invalid on booleans; failure at the root.



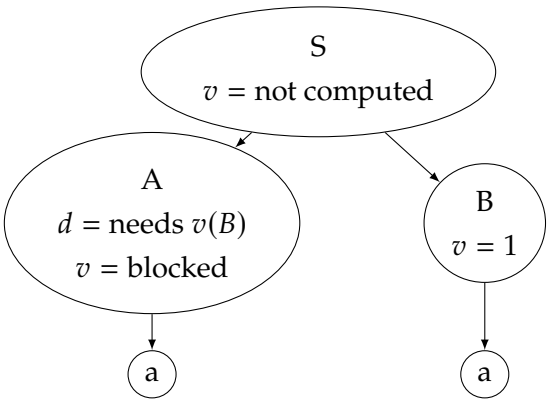
6.7.6 `test_ag5_div_zero` (ag5, tree `pt5`)

Division by zero: $E \to E/F$ with left $F \to 1$ and right $F \to 0$. Failure occurs evaluating $v(E_0) = v(E_1)/v(F)$.



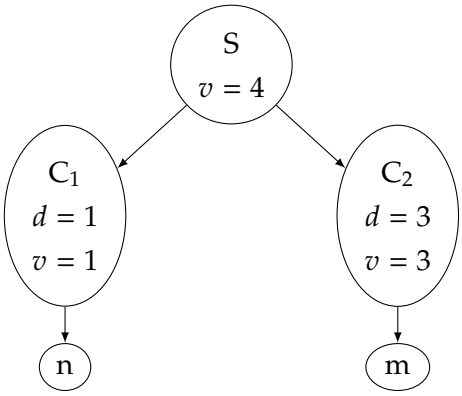
6.7.7 `test_ag6_lattr_violation` (ag6, tree `pt6`)

L-attributed violation at runtime: $S \to A\ B\ \{d(A) = v(B)\}$ requires reading $v(B)$ before B is evaluated (left-to-right discipline). Failure occurs when setting $d(A)$.



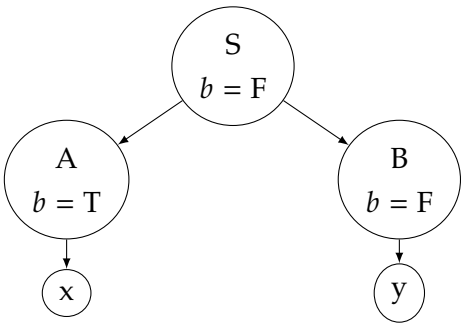
6.7.8 `test_ag7_default_index` (ag7, tree `pt7`)

Default vs. explicit child indices on $S \to C\ C$; inherited $d(C_1) = 1$, $d(C_2) = 3$; $v(S) = v(C_1) + v(C_2) = 4$.



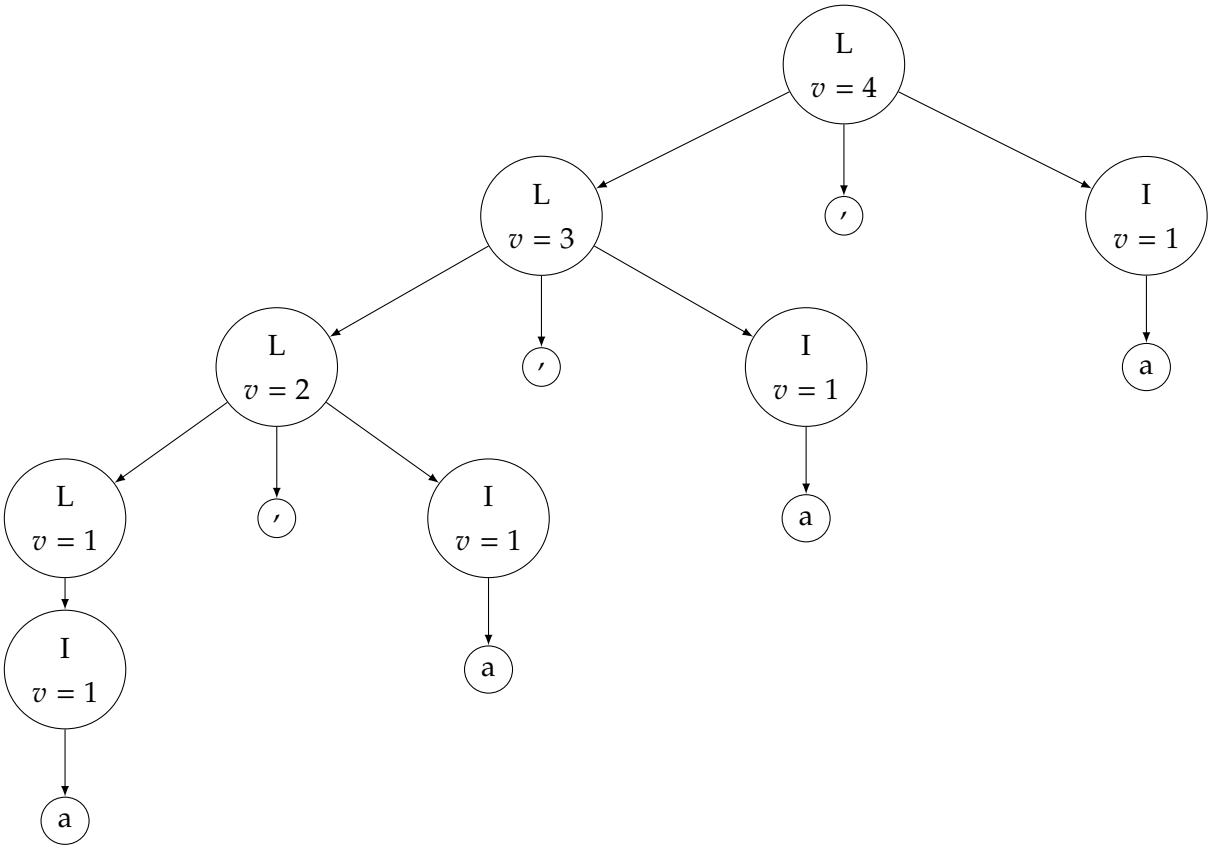
6.7.9 `test_ag8_boolean_flags` (ag8, tree `pt8`)

Boolean synthesized attribute `b` with $A \to x\{b(A) = \mathrm{T}\}$, $B \to y\{b(B) = \mathrm{F}\}$, and $S \to A\,B$ with equality check; $b(S) = \mathrm{F}$.



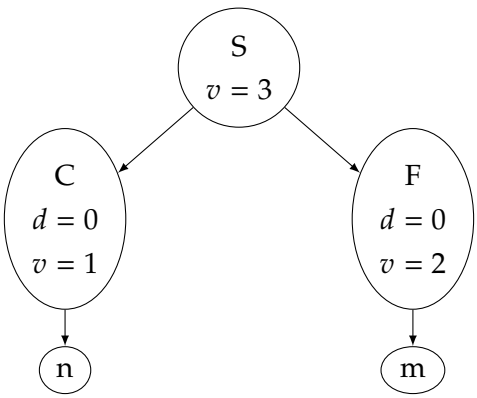
6.7.10 `test_ag9_list_length` (ag9, tree `pt9=make_list 30`)

List length via synthesized `v`. For $n = 30$ the root has $v = 30$. Below is the same left-nested pattern shown for $n = 4$ (illustrative); the test uses $n = 30$.



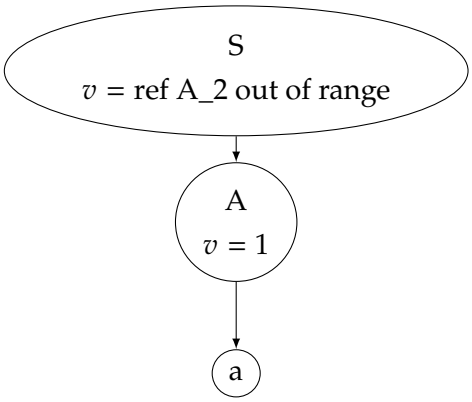
6.7.11 `test_ag10_inherited_default` (`ag10`, tree `pt10`)

Default inherited values $d(C) = 0$, $d(F) = 0$, then $v(C) = 1$, $v(F) = 2$; hence $v(S) = 3$.



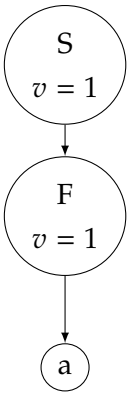
6.7.12 `test_ag11_out_of_range_ref` (ag11, tree `pt11`)

Out-of-range child reference: $S \to A\ \{v(S) = v(A_2)\}$ but only one A child exists. Failure at the root while resolving A_2.



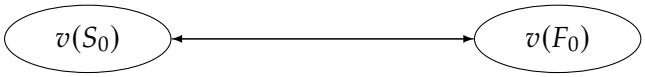
6.7.13 `test_has_cycles_ok` (ok_ag)

Acyclic grammar $S \to F\{v(S) = v(F)\},\ F \to a\{v(F) = 1\}$. No dependency cycle; a concrete tree for the terminal word a is shown. (The unit test itself only checks the cycle predicate.)



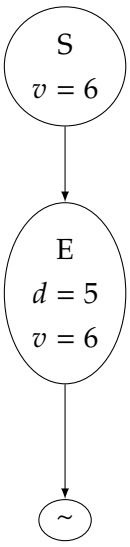
6.7.14 `test_has_cycles_detected` (cyc_ag)

Deliberate attribute cycle: $S \to F\{v(S_0) = v(F_0)\},\ F \to S\{v(F_0) = v(S_0)\}$. No finite parse tree exists; the test inspects the dependency graph (cycle between $v(S_0)$ and $v(F_0)$).



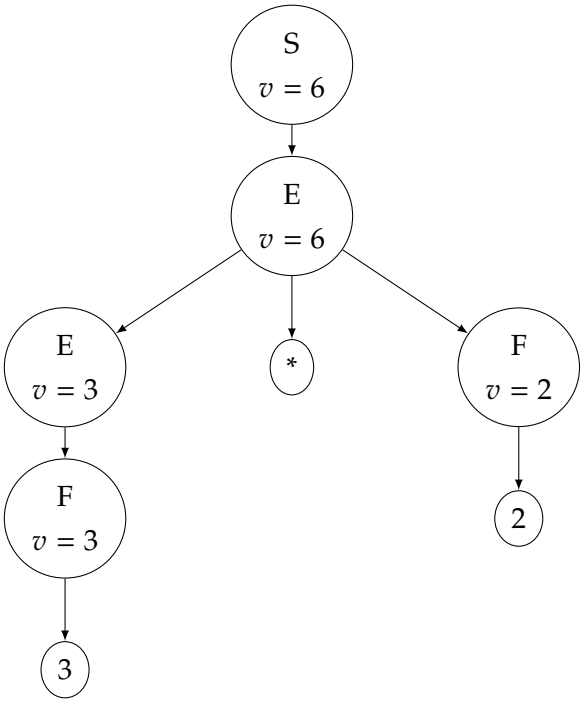
6.7.15 `test_accept_with_tree_ok`

Calls `accept_ag_with_tree` on grammar `ag1` and tree `pt3`. Since `pt3` uses terminals ~ not present in `ag1`, acceptance is expected to be false. For completeness, the same `pt3` is shown below with attributes as they would be under `ag3`. (The unit test only prints the boolean.)



6.7.16 `test_accept_words`

Checks CFG acceptance for the word `32` derived from `ag1`. The test does not evaluate attributes, but a consistent annotated AG tree would have $v(S) = 6$.

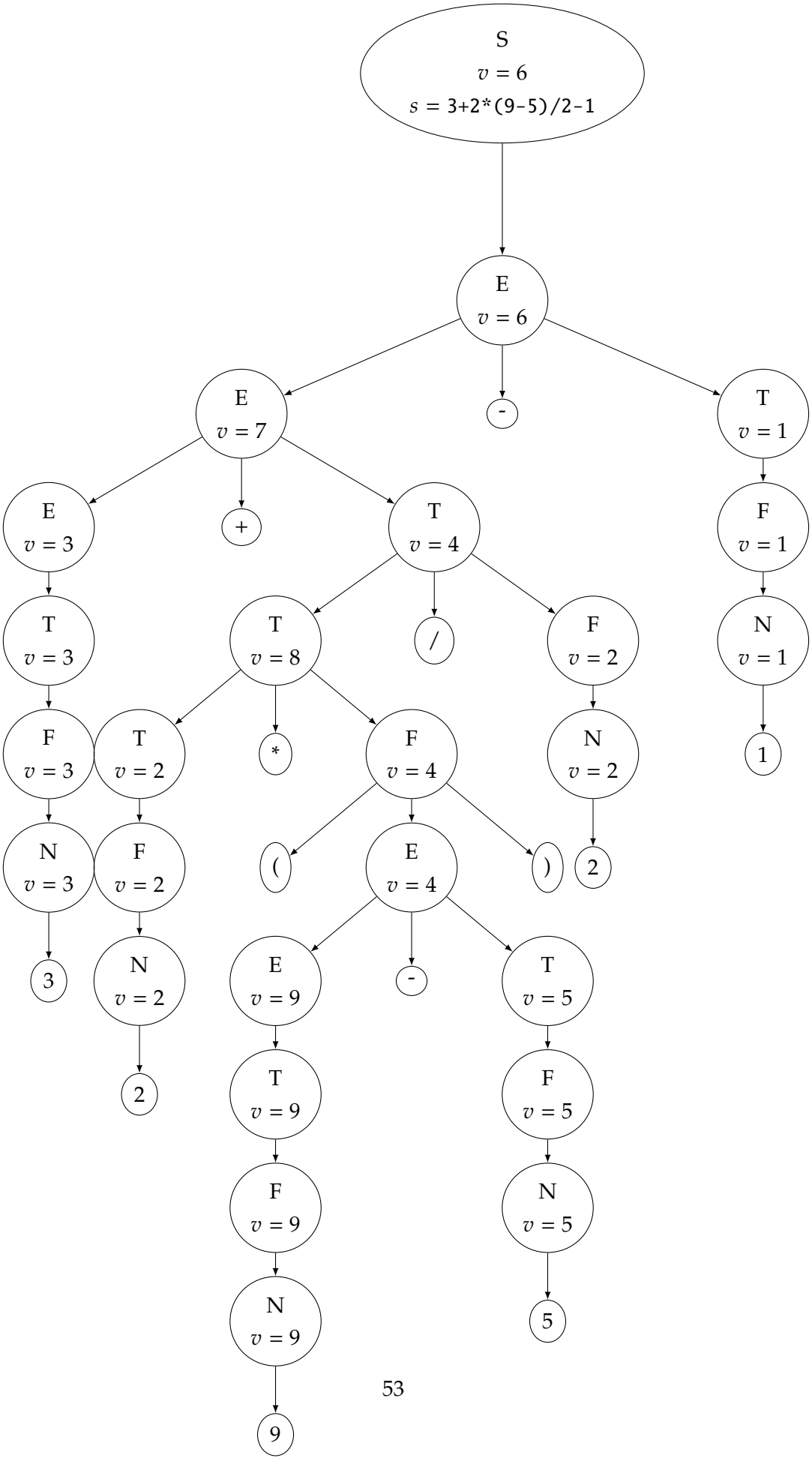


6.7.17 `test_validate`

Runs static validation on the extended arithmetic grammar `ag_expr_ext`. A representative evaluated tree for the fixture `pt_expr_ext` (expression $3 + 2 \times (9 - 5)/2 - 1$) is shown in Section 6.7.18. (Validation itself performs no tree evaluation.)

6.7.18 `test_ag_expr_ext` (`ag_expr_ext`, tree `pt_expr_ext`)

Extended arithmetic with attributes value `v` and pretty string `s`. Root result: $v(S) = 6$, $s(S)$ =`3+2*(9-5)/2-1`. (Height `h` is computed by the grammar but omitted here for brevity.)


S
v = 6
s = 3+2*(9-5)/2-1
E
v = 6
E
v = 7
-
T
v = 1
E
v = 3
+
T
v = 4
F
v = 1
T
v = 3
T
v = 8
/
F
v = 2
N
v = 1
F
v = 3
T
v = 2
*
F
v = 4
N
v = 2
1
N
v = 3
F
v = 2
(
E
v = 4
)
2
3
N
v = 2
E
v = 9
-
T
v = 5
2
T
v = 9
F
v = 5
F
v = 9
N
v = 5
N
v = 9
5
9

7

Evaluation and future work

7.1 Evaluation

During the implementation of the main algorithms of the OCaml-FLAT library, unit tests were written for all operations to provide greater assurance of their correct functioning. Especially on tests for the word acceptance operation, since it is complex and highly visible and calculating attributes, since the calcullation of inherited attributes it was more complex than expected, and it is one of the main focus of this dissertation. Care was also taken to design unit tests that exercise potentially problematic cases, for example attribute grammars that create cyclic situations by continuously changing attribute values. Nevertheless, it would have been preferable to make the application available to a limited number of users so it could be tested more intensively. To help evaluate the quality and correctness of the work, the supervisor of this dissertation was frequently consulted in his capacity as a former lecturer of the Theory of Computation course.

7.2 Conclusion

For this project the OCaml-FLAT library was extended; the OFLAT graphical tool exists to create and manipulate FLAT models, but I did not work on the OFLAT library.

The OCaml-FLAT library was adapted to support attribute grammars, with all necessary types created for their representation and a range of operations implemented. These include evaluation of attribute values for a given input, generation of words of length n, conversions between the various types implemented in the library, and transformations between different grammar models. In addition, functions were implemented to produce derivation trees that represent the history of a derivation or evaluation; these trees are consumed by the graphical tool to perform its main visualization of the evaluation process.

Bibliography

[1] *Chapter 3 ATTRIBUTE GRAMMARS*. Accessed: 2025-01-30. URL: https://homepage.divms.uiowa.edu/~slonnegr/plf/Book/Chapter3.pdf (cit. on p. 8).

[2] N. Chomsky. "Three models for the description of language". In: *IRE Transactions on Electronic Computers* EC-8.1 (1956) (cit. on pp. 3–5).

[3] B. Demange et al. *From Bytecode to JavaScript: The Js of ocaml Compiler*. 2012. URL: https://citeseerx.ist.psu.edu/document?doi=1b87d68c7c4f7b897eeb09804657225f6f8f762f&repid=rep1&type=pdf (cit. on p. 24).

[4] C. Gordon. *The Circularity Problem for Attribute Grammars*. Accessed: 2025-01-17. 2000. URL: https://courses.cs.washington.edu/courses/cse531/00au/gordon.pdf (cit. on pp. 10, 11).

[5] D. E. Knuth. "Semantics of Context-Free Languages". In: *Theory of Computing Systems* 2.2 (1968), pp. 127–145 (cit. on pp. 6, 7, 10–12).

[6] J. M. Lourenço. *The NOVAthesis LaTeX Template User's Manual*. NOVA University Lisbon. 2021. URL: https://github.com/joaomlourenco/novathesis/raw/main/template.pdf (cit. on p. i).

[7] *OCaml History*. Accessed: 2025-01-17. URL: https://ocaml.org/about (cit. on p. 23).

[8] *OFLAT*. Accessed: 2025-01-30. URL: http://ctp.di.fct.unl.pt/FACTOR/OFLAT/ (cit. on p. 20).

[9] A. Ravara. *Slides da disciplina de teoria da computação*. Rel. téc. FCT-UNL, 2020 (cit. on p. 12).